

HarnessX：可组合、自适应、可演化的 Agent Harness Foundry

原文：/Users/sky/Desktop/twitterfree/2606.14249v1.pdf arXiv：2606.14249v1
[cs.AI]，2026-06-12 作者：Darwin Agent Team

说明：本文为中文译文。公式、模型名、数据集名、代码标识符、文件路径和引用编号保持原样；图内英文文字已在对应图注下翻译为中文。参考文献保留原始书目信息，避免破坏可检索性。

摘要

AI agent 的性能在很大程度上取决于运行时 harness：也就是提示词、工具、记忆和控制流这一整套中介机制，它决定模型如何观察、推理和行动。然而，当下的 harness 仍然主要是手工制作且静态的：每换一个新模型或新任务，仍然需要定制脚手架；执行过程中产生的大量轨迹也很少被提炼成系统性的改进。

我们提出 HarnessX，一个用于构建可组合、自适应、可演化 agent harness 的 foundry。HarnessX 通过替换代数把类型化 harness primitive 组装起来；通过 AEGIS 对其进行自适应，AEGIS 是一个由轨迹驱动的多 agent 演化引擎，其基础是符号适应和强化学习之间的“操作镜像”；并且通过把轨迹同时转化为 harness 更新和模型训练信号，闭合 harness-model 循环。

在五个 benchmark 上，ALFWorld、GAIA、WebShop、 τ^3 -Bench 和 SWE-bench Verified，HarnessX 带来平均 +14.5% 的提升，最高 +44.0%；基线越低，提升越大。这些结果说明，agent 进步不必只来自模型规模扩张：从执行反馈中组合并演化运行时接口，是一条可操作、互补的杠杆。完整代码库将在未来开源。

图 1 : HarnessX 总览

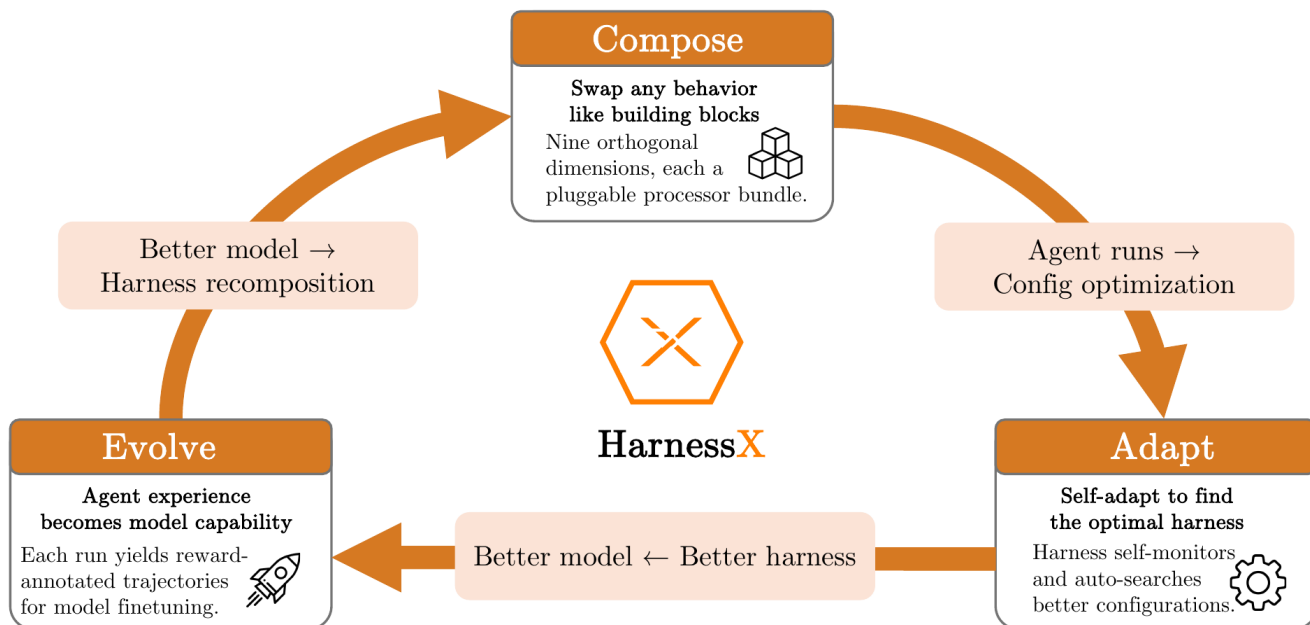


图 1 原图；图内英文已在下方表格翻译为中文。

图内文字翻译：

原文	中文
Compose	组合
Swap any behavior like building blocks	像搭积木一样替换任意行为
Nine orthogonal dimensions, each a pluggable processor bundle.	九个正交维度，每个维度都是可插拔的 processor bundle。
Better model -> Harness recomposition	更好的模型 -> Harness 重新组合
Agent runs -> Config optimization	Agent 运行 -> 配置优化
Evolve	演化
Agent experience becomes model capability	Agent 经验转化为模型能力
Each run yields reward-annotated trajectories for model finetuning.	每次运行都会产生带奖励标注的轨迹，用于模型微调。
Better model <- Better harness	更好的模型 <- 更好的 harness
Adapt	适应
Self-adapt to find the optimal harness	自适应搜索最优 harness
Harness self-monitors and auto-searches better configurations.	Harness 自我监控，并自动搜索更好的配置。

目录

1. 引言
2. 相关工作
3. Harness 组合
4. Harness 适应

5. Harness-Model 协同演化

6. 实验

7. 讨论

8. 结论

附录 A-E

1. 引言

现代 agent 的能力不仅取决于底层模型，也取决于周围 harness 所施加的中介机制。这个 harness 决定任务如何被表示、外部服务如何被访问、执行过程中的中间决策如何被传达，从而把模型的原始输出转化为结构化的 agent 行为。当 agent 开始处理更长周期、更复杂环境中的任务时，harness 设计就成为 agent 开发的核心组成部分。

尽管如此，harness 开发仍远远不是一门成熟的工程学科。第一，harness 是手工工程化且静态的：模型版本、工具或问题域的任何变化，都需要定制修改，缺乏基于经验自动改进的机制。第二，harness 在架构上高度纠缠：提示词模板、工具 wrapper、重试策略和记忆通常混在同一批代码路径中，因此一个组件的改动会悄悄破坏另一个组件；跨领域复用也退化为复制粘贴，而不是组合。第三，harness 工程和模型训练相互独立：为了改进 harness 而收集到的轨迹数据被丢弃，而不是纳入模型训练；模型的改进也不会转化为 harness 的改进。

我们通过把 harness

视为一个一等对象来弥补这些缺口：它可以被组合、适应，并且与模型一起演化。HarnessX 把这一原则实现为统一的 harness foundry。它首先建立模块化基础：覆盖 context、tools、skills、control 和 memory 的 harness primitive 通过类型化接口描述，并通过替换代数组合。这把现有系统通常混在一起的关注点拆开。在这个底座之上，我们提出 AEGIS，一个由可观测性驱动、可审计的 harness 适应引擎。把 harness 适应视为一个作用于符号 artifact 的学习问题，而不是临时编辑问题，会暴露出标准强化学习病理，reward hacking、catastrophic forgetting 和 under-exploration，会转化为具体设计风险。为应对这些风险，AEGIS 结合完整轨迹可观测性和四阶段 pipeline：Digester、Planner、Evolver 和 Critic，分别用于压缩轨迹、规划适应、生成候选方案、评估变更。最后，我们通过 harness-model 协同演化闭合 harness 适应和模型训练之间的循环。harness 适应中产生的轨迹作为模型训练的强化学习信号，使模型改进反馈到后续 harness 演化中。

我们在五个 benchmark 上经验验证

HarnessX：GAIA、ALFWorld、WebShop、 τ^3 -Bench、SWE-bench Verified；覆盖三个 task-agent 家族：Claude Sonnet 4.6、GPT-5.4、Qwen3.5-9B；最多运行 15 轮演化。Harness 演化在 15 个 model-benchmark 配置上带来平均绝对提升 +14.5%，在有提升的配置中，单项提升从 0.0% 到 +44.0% 不等，14/15 个配置有提升，从接近天花板基线的 τ^3 -Bench +1.1%，到最弱 agent 的 ALFWorld +44.0%。提升呈现反向 scaling 模式：在 ALFWorld 和 GAIA 上，最弱的 task agent 受益最大，例如 ALFWorld 上 Qwen3.5-9B 提升 +44.0%，而 Sonnet 4.6 提升 +11.2%。这说明演化出的 harness 填补了弱模型无法自我纠正的行为缺口。在异质任务集 GAIA 上，单 harness 演化会停滞；variant isolation 消融恢复了稳定提升，15 轮内提升 +13.6%，且无退化。

贡献概括如下：

- **Harness Composition**。我们把 harness 形式化为一等、类型化对象，由挂载到生命周期 hook 的 processor 组成。九维分类法覆盖完整行为空间，替换代数支持按任务进行类型安全的插入和移除。这种组合结构使每个行为变更的预期作用范围显式化，这是稳定演化所需 variant isolation 的前提。
- **Harness Adaptation**。我们提出 AEGIS，一个由轨迹驱动的多 agent harness 演化引擎。操作镜像把 harness 适应映射到标准 RL 构造，把熟悉的 RL 病理，reward hacking、catastrophic forgetting、under-exploration，转化为具体设计风险，并由带确定性 gating 的四阶段 pipeline 处理。可选 variant-isolation 策略能防止异质 benchmark 上的跨任务干扰。
- **Harness-Model Co-Evolution**。我们通过在共享 replay buffer 上交错进行 harness 演化和模型强化学习，闭合优化循环。Cross-harness GRPO 使模型能够内化 successive harness versions 中的策略，突破 harness-only 适应的 scaffold ceiling 和 model-only RL 的 training-signal ceiling。
- **Empirical Validation**。跨五个 benchmark、三个 task-agent 家族、最多 15 轮演化，HarnessX 带来平均 +14.5% 的提升，最高 +44.0%，且基线越低提升越大。variant-isolation 消融解决了异质任务集上的停滞，协同演化在 harness-only 演化之上额外提升 +4.7%。

2. 相关工作

2.1 Harness 工程

现有 agent 基础设施处在一条逐渐“更有主见”的 harness 抽象光谱上。在 primitive 层，LangChain、LlamaIndex、Smolagents 这类库提供了提示词、工具、检索和记忆的类型化构建块。这些 primitive 可以被单独测试，但不支持 harness 级组合：两个使用相同 primitive 构建的 harness，结构仍然可能不同。

下一层抽象把这些 primitive 编排成可复用模式。LangGraph 用有状态图建模 agent 行为；AutoGen 把多 agent 交互建模为结构化对话；CrewAI 给 agent 分配基于角色的身份；Letta 把自主循环和持久记忆结合起来。虽然这些框架让 harness 编写更容易，但它们施加了特定控制循环，因此模式组合、组件替换、跨任务移植增强，主要仍是手工工作。

最后，还有产品化、领域特定的 harness，例如 Claude Code、Cursor、Manus 和 DeerFlow。这些系统展示了 harness 设计的影响力，但架构上仍然静态，只能通过人工迭代演化。

在这三层中，有两个结构性缺口持续存在。第一，没有一层把 harness 暴露为由类型化元素组成、可替换的实体，因此构建按任务定制的 harness 总是涉及重写。第二，不存在循环内改进机制：harness 一旦定义，就只能在人类发布周期之间迭代。

同期，Claude Code 引入了 Dynamic Workflows，使模型能够在运行时生成任务特定的 harness 脚本。这是朝自适应 harness 前进的一步，但它仍局限在单个 session 内，没有持久的基于轨迹的优化、跨 session 演化或 harness-model 协同训练。HarnessX 同时解决两个缺口：把 harness 适应视为多轮、轨迹驱动的学习问题；用类型化组合支持 variant isolation，用结构化可观测性检测病理，并用共享 replay buffer 闭合 harness 演化和模型训练之间的循环。

2.2 自演化 Agent

自演化 agent 研究关注 agent 系统如何在不重新训练底层 foundation model 的情况下改进。早期工作集中在最容易编辑的单一方面：提示词。APE、OPRO、EvoPrompt、Promptbreeder

把指令表述视作黑盒优化问题；ProTeGi 和 TextGrad

引入受梯度启发的文本反馈，使优化过程显式化。DSPy 和 MIPRO 通过编译声明式 LM program 扩展了这种方法，并用标注数据优化其提示词。这些方法确立了“指令是可学习组件”这一事实，但 harness 级特征，工具、记忆、控制流，仍然在优化范围之外。

另一条工作线通过积累并复用先前执行经验来改进 agent：Memento 用 case-based memory 改进 agent，而不微调模型；MIA 在一个 Manager-Planner-Executor

框架中统一非参数和参数记忆：一边是压缩轨迹的非参数存储，另一边是测试时即时演化的参数 planner，两者通过双向循环不断转换经验，并在十一个 benchmark 上显示优势。

后续工作把优化扩展到 agent

workflow。GPTSwarm、ADAS、AFlow、A²Flow、AgentSwift、ResMAS 和 EvoAgentX

搜索协作策略、agent 顺序和聚合机制。这些工作证明 workflow 结构是可学习的，而且比 prompt-only 优化带来更大收益。但组件级 artifact，工具实现、记忆策略、节点内部提示词，仍然是静态的：优化范围覆盖的是组件间关系，而不是完整 harness。

最后一组工作明确把 harness 作为演化目标。SICA 直接优化 SWE-bench agent 的源码；Darwin Gödel Machine 提出在 agent 变体数据库上做开放式优化。HyperAgents

让优化过程本身自适应；Meta-Harness 通过基于文件系统的接口提升采样效率。AHE 和 Life-Harness 强调可观测性、可解释性和源码重写。总体而言，这些工作把 harness 确立为演化目标，并证明可观测性对于稳定自改进至关重要。然而，它们缺乏一个把观测到的故障模式连接到原则化防御的统一理论框架。

Heuristic-learning theory 部分弥补了这个缺口，它把 RL

概念映射到符号自优化更新。在这个框架中，可观测轨迹对应正确的 credit assignment，可证伪 change manifest 对应 reward shaping，proposal-critique cycle 提供结构化探索。HarnessX 实例化这一范式，把这种对应关系形式化为 RL 和符号 harness 演化之间的操作镜像。

3. Harness 组合

相关工作中识别出的缺口，是缺少一个把 harness 暴露为类型化、可替换实体的基础设施层。Primitive 库把组合留给应用代码；编排器暴露固定模式集合；产品级 harness

是端到端黑盒。没有组合底座，每个行为变更或跨团队交接都需要重新实现。HarnessX

用统一设计原则解决这一点：harness 是一等值，processor 是类型化原子组件，组合通过在类型化 hook point 插入 processor 完成。我们形式化 harness、它的构建块 processor，以及九维 processor 分类法。这些定义刻意保持简洁：它们的作用是建立词汇，并暴露 harness 演化所操作的编辑面。

表 1：Hook point 及允许修改

Hook	Event type	允许修改
task_start	TaskStartEvent	system prompt
step_start	StepStartEvent	结构化历史编辑
before_model	BeforeModelEvent	最后一条用户内容；追加一条用户消息
after_model	ModelResponseEvent	response 内容、tool calls
before_tool	ToolCallEvent	tool input、approval flag
after_tool	ToolResultEvent	tool result
step_end	StepEndEvent	只读
task_end	TaskEndEvent	只读

3.1 Harness 作为一等对象

HarnessX 中的 harness 是一对 $H=(M,C)$ ，其中 M 是模型配置， C 是 harness 配置。两者处理互不重叠的关注点： M 记录哪个模型服务哪个角色，例如 main、judge、evaluator，以及每个角色的 fallback policy； C 记录 agent 如何行为，并独立于模型身份。两者通过 `agent = model_config.agentic(harness_config)` 组合成可执行 agent：HarnessX 中的 agent 是绑定到模型的 processor pipeline，二者都可以独立替换。

Harness 配置本身分解为

$C=(P,S)$ 。 $P:\text{Hook}\rightarrow\text{List}[\text{Processor}]$ 是以 hook 为索引的 processor 列表， Hook 是表 1 中的八个生命周期事件。 S 是一组固定、正交的 slot resources：tool registry、tracer、workspace、sandbox provider 和 plugin list。Slot 是单例，在一个配置中的所有 processor 间共享；processor state 则是实例私有的。 P 实现所有 per-step 行为； S 容纳 processor 依赖但不拥有的共享基础设施。

我们称 C 为一等对象，因为它可以独立序列化、比较、哈希和替换。两个 agent 共享 C 但 M 不同，会执行相同的 processor pipeline，行为差异只来自模型响应；两个 agent 共享 M 但 C 不同，则行为上不同。这种 reification 是程序化演化的前提。

3.2 Processor 抽象

HarnessX 中每一种 per-step 行为都实现为一个 processor：满足协议 `async def process(self, event: Event) -> AsyncIterator[Event]` 的对象。processor 消费一个事件，产出零个或多个事件，并且恰好产生五种结果之一：pass-through，原样 yield；transform，yield 修改后的事件；split，yield 多个同类型事件并在下游独立处理；intercept，不 yield，从而阻断传播；interrupt，抛出异常并终止循环。这个受限接口使组合成为可能：同一 hook 上的每个 processor 都消费并产出同一种事件类型，因此 processor 可以顺序组合，也可以插入或移除，而不会影响周围 pipeline 的类型正确性。

如表 1 所示，processor 挂载到 run loop 发出的八个 hook point 之一。run loop 在每次调用后验证 hook contract：任何违规，比如修改只读字段，都会立即抛出异常，而不是悄悄传播损坏状态。每个 processor 携带三个用于控制组合的类级元数据字段：`_singleton_group` 命名互斥类，确保每组最多一个

processor; `_order` 是 hook 内排序提示，使用 PRE、NORMAL、POST 等常量; `_after` 是对其他 singleton group 的软依赖列表。

这种设计使 harness 演化成为一等操作：AEGIS 可以在特定 hook 插入新 processor，通过匹配 singleton group 替换现有 processor，或者完全移除某个 processor，而无需触碰相同或不同 hook 上的其他 processor。由于类型 contract，输入事件类型等于输出事件类型，在每个 hook 上被强制执行，任何这种替换都会保持整体 pipeline 的 well-typedness。元数据字段进一步约束组合：`_singleton_group` 防止冲突副本，`_order` 保证新插入 processor 与现有 processor 可预测地交互。这些保证就是 variant isolation 的机制基础：每个变体只在“哪些 processor 占据哪些 hook”上不同，类型系统保证没有变体能在演化过程中悄悄违反 pipeline contract。

3.3 九维分类法

我们沿九个维度组织行为空间：model selection (D1) 决定哪个模型服务哪个角色；context assembly (D2) 决定每一步呈现给模型什么；memory management (D3) 管理跨 step 和跨 session 保留什么；tool ecosystem (D4) 控制 agent 可以调用哪些工具；execution environment (D5) 决定工具引发的副作用在哪里发生；evaluation and reward (D6) 指定如何评判结果；control and safety (D7) 强制规则，防止 agent 循环、超支或偏离意图；observability (D8) 记录每个事件、模型调用和工具调用；training bridge (D9) 把执行轨迹转化为强化学习记录。图 2 展示了完整分类法、代表性 processor，以及它们在标准配置中通常挂载的 hook。

实践中，AEGIS 演化时的编辑跨越全部九个维度：D2 context assembly 和 D4 tool ecosystem 是最频繁的编辑目标；D8 observability 提供 AEGIS 自身推理的轨迹底座；D9 training bridge 为协同演化提供轨迹记录，从而闭合优化循环。

表 2：操作镜像

RL 概念	符号空间对偶	AEGIS 实现
Policy π	Harness-update procedure π_{evo}	四阶段 pipeline
State s_t	$\mathcal{H}_t, \mathcal{T}_t$	Harness 配置 + trace store
Action a_t	类型化 harness edit	Builder operation + change manifest
Feedback	Trace τ + verifier score r	Observability layer
Update	$\mathcal{H}_{t+1} \leftarrow \text{U}(\text{widetilde{e}}(\mathcal{H}_t, \mathcal{T}_{t,r}))$	确定性 acceptance gate

4. Harness 适应

组合层提供类型化、可替换的 harness；如图 2 所示，AEGIS 是演化这个 harness 的系统。关键洞见是，harness 演化在结构上可映射到符号空间中的强化学习：harness 配置是 state，类型化编辑是 action，执行轨迹加 verifier score 构成 feedback。这个映射具有预测性：它识别出三种类似已知 RL 病理的失败模式，reward hacking、catastrophic forgetting 和 under-exploration；这些失败模式推动 AEGIS 的架构防御，并在实验中得到确认。

我们先形式化这种对应关系，接着分析它预测的病理，再把四阶段 pipeline 推导为防御架构，随后给出适应循环，并引入 variant isolation，用于稳定的多变体演化。

图 2 : AEGIS 演化循环

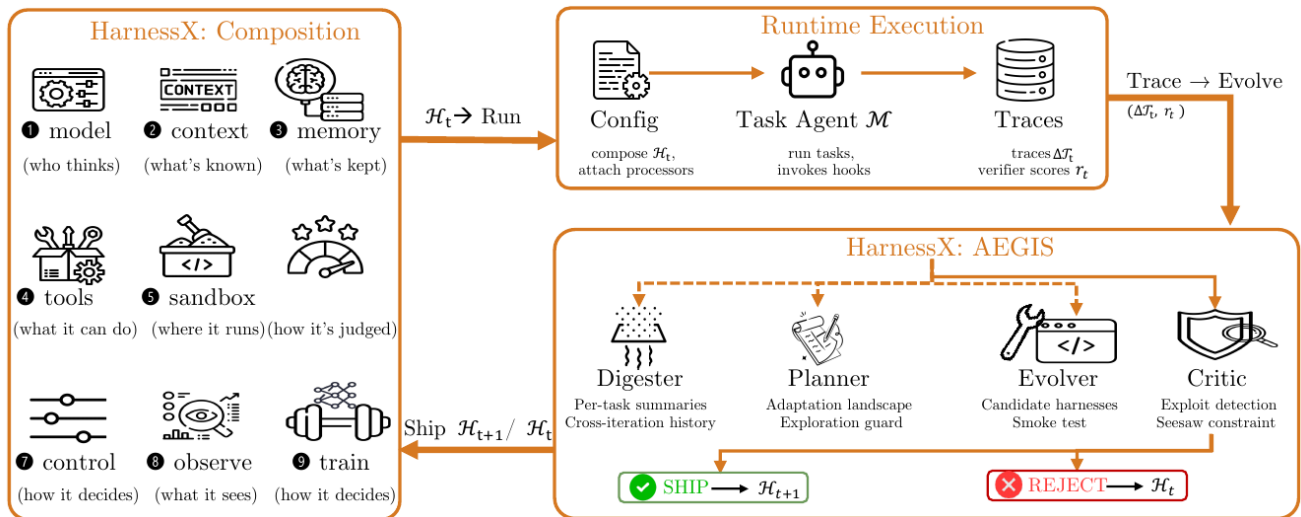


图 2 原图；图内英文已在下方表格翻译为中文。

图内文字翻译：

原文	中文
HarnessX: Composition	HarnessX : 组合
Runtime Execution	运行时执行
Trace -> Evolve	轨迹 -> 演化
model (who thinks)	模型 (谁在思考)
context (what's known)	上下文 (已知什么)
memory (what's kept)	记忆 (保留什么)
tools (what it can do)	工具 (能做什么)
sandbox (where it runs)	沙箱 (在哪里运行)
how it's judged	如何被评判
control (how it decides)	控制 (如何决策)
observe (what it sees)	观测 (看见什么)
train (how it decides)	训练 (如何学会决策)
Config compose, attach processors	配置组合，挂载 processor
Task Agent run tasks, invokes hooks	Task Agent 运行任务，调用 hook
Traces, verifier scores	轨迹、verifier 分数
Digester	消化器
Per-task summaries	按任务摘要
Cross-iteration history	跨迭代历史
Planner	规划器
Adaptation landscape	适应地形
Exploration guard	探索保护
Evolver	演化器
Candidate harnesses	候选 harness
Smoke test	冒烟测试
Critic	批评器
Exploit detection	利用漏洞检测

原文	中文
Seesaw constraint	跷跷板约束
SHIP / REJECT	发布 / 拒绝

4.1 操作镜像

我们把 harness 演化形式化为作用于符号 artifact 的 MDP。表 2 总结了映射；先给出三个定义来支撑这种对应关系。

定义 1：Harness Configuration。Harness 配置是一个元组

$\mathcal{H}=(c_1, c_2, \dots, c_9)$ ，其中每个 $c_i \in \mathcal{C}_i$

实例化九个行为维度之一：model selection、context assembly、memory management、tool ecosystem、execution environment、evaluation and reward、control and safety、observability、training bridge。每个 $\mathcal{C}_i$ 是维度 i 的有效 processor 配置集合，受 hook-type contract 和 singleton-group exclusion 约束。

定义 2：Harness Edit。Harness edit 是函数

$e:\mathcal{H} \rightarrow \mathcal{H}$ ，修改一个或多个维度，同时保持类型 contract。Action space $\mathcal{E}$ 是离散但开放的：每个 edit 都是由 meta-agent LLM 生成的代码级 artifact，例如新 processor 源码、修改后的 prompt template、重新配置的 tool registry 或 control-flow rewrite，而不是从预枚举集合中选择。组合爆炸不是靠穷举搜索控制，而是靠 LLM 的生成能力，Planner 从基于轨迹的假设中提出 edit，并靠类型约束在生成阶段剪掉无效组合。

定义 3：Operational Mirror。操作镜像是元组

$(\mathcal{H}, \mathcal{E}, \mathcal{R}, \mathcal{T})$ ，其中 $\mathcal{H}$ 是 harness-configuration space，也就是 state； $\mathcal{E}$ 是代码级 edit space，也就是 action； $\mathcal{R}:\mathcal{H} \times \mathcal{E} \rightarrow \mathbb{R}$ 把一个 configuration-edit pair 映射为标量 reward，即 adaptation batch 上聚合的 verifier score； $\mathcal{T}$ 是 trace store，提供标量信号之外的结构化 feedback。这个元组在 harness 层形成一个 MDP：harness 配置是 state，类型化 edit 是 action，执行轨迹加 verifier score 构成 feedback，确定性 acceptance gate 控制 state transition。

MDP 实例化。令 $\mathcal{H}_t$ 表示 iteration t 的 harness 配置，演化过程中模型 $\mathcal{M}$ 固定；令 $\mathcal{T}_t$ 表示所有先前执行累计的 trace store。符号 state 定义为 $s_t=(\mathcal{H}_t, \mathcal{T}_t)$ 。Harness-update policy π_{evo} 选择 action $a_t \sim \pi_{\text{evo}}(\cdot | s_t)$ ，其中 $a_t \in \mathcal{E}$ 是从 builder algebra 中抽出的代码级 edit。应用该 edit 得到候选 harness $\widetilde{\mathcal{H}}_t=a_t(\mathcal{H}_t)$ 。用固定模型 $\mathcal{M}$ 在 adaptation batch 上运行候选 harness，产生新轨迹 $\Delta \mathcal{T}_t$ 和逐任务 verifier score r_t 。确定性 acceptance operator $U(\widetilde{\mathcal{H}}_t, \mathcal{T}_t, r_t)$ 随后要么提交候选，要么拒绝它，并强制 seesaw constraint：候选不能让 $\mathcal{T}_t$ 中记录的任何先前已解任务退化。无论提交还是拒绝，trace store 都会增长： $\mathcal{T}_{t+1}=\mathcal{T}_t \cup \Delta \mathcal{T}_t$ 。

这个 MDP 在 harness 层运行：在单个任务内部， $\mathcal{H}_t$ 和固定 $\mathcal{M}$ 共同决定 agent 行为；跨 iteration，harness-update policy π_{evo} 修改 harness。AEGIS 把 π_{evo} 实现为四阶段 pipeline：Digester、Planner、Evolver、Critic，它通过轨迹压缩、适应规划、编辑生成、候选评估，把 s_t 映射到候选 edit。

4.2 符号空间中的病理

镜像不只是类比；它把强化学习概念转化为设计要求。我们把 RL 中三种文献充分讨论的失败模式，reward hacking、catastrophic forgetting、under-exploration，统称为 RL pathologies。一旦把 harness 适应建模为符号 artifact 上的

MDP，这些病理会以被放大的形态重新出现，原因是符号环境有两个性质：第一，语言模型 evolver 可以构造数值参数扰动无法表达的结构化 exploit；第二，对共享组件的 edit 会通过 harness 非局部传播。下面每个病理都对应 AEGIS 架构中的一种防御。

Reward hacking。 在标准 RL 中，reward hacking 是利用 reward signal 的漏洞，而不真正完成任务。符号 harness 演化放大了这种风险，因为 evolver 可以直接瞄准验证协议：把 benchmark 答案嵌入 prompt，利用 verifier 的格式规律，或者引入一个 processor，把输出重写成符合 verifier 期望的形式。

Catastrophic forgetting。 Catastrophic forgetting 是指在任务分布某一区域上的性能提升损害另一区域。在符号 harness 演化中，修复故障模式 A 的 edit 可能悄悄让模式 B 退化，因为影响会通过共享 context、tools、memory policy 和 control rule 传播。没有显式 regression checking 时，只根据失败任务轨迹进行条件化的 evolver 无法区分局部增益和全局退化。

Under-exploration。 Under-exploration 表现为偏向低风险局部 edit：prompt 重写、tool description 微调、轻量 control-flow 调整。这些 edit 生成成本低，且经常能通过 gating 而不退化已解任务，导致后续 Planner 假设继续偏向同一 edit neighborhood。结构性变化，例如把一个 agent 拆成多个、替换控制策略、采用新的 memory architecture，需要刻意形成假设，很少从基于轨迹的局部修复中自然出现。如果没有机制提出超出当前失败邻域的 edit，系统会在局部 edit 用尽后停滞。

小结。符号 harness 演化继承了 RL 的结构性风险，AEGIS 分别用 Critic 防御 reward hacking，用确定性 gating layer 防御 catastrophic forgetting，用 Planner 防御 under-exploration。

Algorithm 1：AEGIS Harness 演化循环

输入：初始 harness $\mathcal{H}_0$ 、meta-agent $\mathcal{M}$ 、预算 B 、patience P 、阈值 α 。输出：演化后的 harness $\mathcal{H}_{t+1}$ 、trace store $\mathcal{T}_{t+1}$ 。

1. 初始化 $\mathcal{T}_0 \leftarrow \emptyset$ ， $\text{idle} \leftarrow 0$ 。
2. 对每轮 $t=0,1,\dots,T-1$ ：
 - 抽样 batch B_t ；运行 $\mathcal{H}_t$ 得到 traces $\Delta\mathcal{T}_t$ ；更新 $\mathcal{T}_{t+1}$ 。
 - Digester（选择性）：从本轮 traces 和历史 trace store 生成证据与

actionability。如果 $a_t < \alpha$ ，本轮 no-op， $idle++$ 。- Planner（选择性）：从证据生成 adaptation landscape。如果为空，本轮 no-op。- Evolver（选择性）：生成 K_t 个候选 harness 及 manifest。如果没有候选，本轮 no-op。- Critic & Gate（强制）：Critic 排序候选；确定性 gate 按序测试候选，第一位通过者被提交。- 如果没有候选通过，则保持旧 harness 并 $idle++$ ；若 $idle \geq P$ ，停止。

3. 返回 $\mathcal{H}_{t+1}, \mathcal{T}_{t+1}$ 。

4.3 AEGIS 架构

AEGIS 是 HarnessX 的 harness-evolution engine。它由四个阶段组成，按预定义 workflow 排列：Digester、Planner、Evolver、Critic；全部由同一个 meta-agent LLM 驱动，并由它选择性调用。没有外部 router 决定阶段执行；meta-agent 自己在每个阶段判断是否有足够信号继续。Digester、Planner、Evolver 各自评估 continuation condition，并可能让本轮短路，例如 actionability 低于阈值、landscape 为空、没有可行候选。Critic 和确定性 gating layer 对进入它们的候选是强制的。任何 edit 没有经过 Critic 和 gate 都不能 ship。

阶段分工对应 4.2 节中的病理：Digester 把原始轨迹压缩为结构化 task-level evidence；Planner 构建覆盖增量变化和结构变化的 adaptation landscape；Evolver 生成类型化 builder edit 和显式 change manifest；Critic 加 gating layer 拒绝缺少轨迹支撑的改进声明，或会让先前已解任务退化的 edit。

所有阶段共享一个信息底座：trace store。它是执行事件、verifier-scored outcome、regression signal，以及已 ship 或 rejected edit 的结构化记录。除 trace store 和当前 harness $\mathcal{H}_t$ 外，没有阶段消费其他输入。数据通过带选择性 gating 的 pipeline 前向流动：Digester 可判定没有可执行故障并立即终止；Planner 可判定当前证据和 edit 历史下没有可行 landscape；Evolver 可产出零个类型安全候选。在这些情况下，本轮都会干净地以 no-op 退出。只有 Critic 和确定性 gate 是无条件的：任何上游幸存候选都必须通过二者才能 ship。Critic 还可以在最终裁决前向 Evolver 发出一次 revision request。

Digester。GAIA 单轮运行，103 个任务、pass@2，会产生约 1000 万 token 的原始轨迹，包括模型推理步骤、工具调用及输出、时间元数据。直接把这些内容传给下游阶段会超过上下文限制，而朴素截断会丢掉诊断信号。Digester 把每个任务的轨迹压缩成结构化 per-task summary：二值结果、失败类别、牵涉组件 identifier、支持性证据摘录。它还提供跨迭代连续性：每个任务摘要连接其先前结果和 shipped edit 历史，使 Planner 能区分持久失败和瞬时噪声。

Planner。Planner 接收 Digester 输出，也就是带跨迭代历史的 task-level summary，并构建 adaptation landscape：哪些任务失败、尝试过哪些 edit、牵涉哪些组件、哪些 edit 类型仍未尝试，例如 prompt、tool、processor、configuration。这一阶段是抵抗 under-exploration 的主要防线：在 edit 生成前构建 landscape，防止 pipeline 收敛到基于轨迹的局部修复，确保工具添加、processor 重写、memory-policy redesign 等结构性变化会和增量 prompt edit 一起被考虑。

Evolver。给定 Planner 的 adaptation landscape，Evolver 生成一个或多个候选 harness $\{\tilde{\mathcal{H}}_t^{k=1} \wedge K_t\}$ ，每个候选都表示为对当前 harness $\mathcal{H}_t$ 的类型化 builder operation。每个候选都携带 change

manifest：编辑过的组件、预期行为效果、预计会改进或退化的任务。引入新的 processor code 时，Evolver 还必须提供 smoke test，确认 processor 能在合成输入上实例化并运行而不抛异常。Builder algebra 保证类型安全，但不保证行为安全；一个 type-check 的 edit 仍可能产生非局部行为影响，只能由 Critic 和 gating layer 检测。

Critic 和 gating。Critic 防御 reward hacking；确定性 gating layer 防御 catastrophic forgetting。Critic 通过比较 change manifest 和 trace evidence 评估候选，并判断 edit 是否会通过 shared state 或 control flow 造成非局部风险。发现缺口时，它向 Evolver 发出一次 revision request。最多一轮 revision 后，Critic 返回 no_op 或有序 ship_ranking。随后确定性 gate 按序执行 acceptance check：manifest 完整性、配置归一化、build 或 smoke test，以及 seesaw constraint。第一个失败检查会终止序列；通过的候选被提交，失败候选以 rejection reason 归档。这把 LLM 判断和 acceptance 解耦：无论 Critic 如何建议，只有确定性检查决定 ship。

设计原则。语言模型 subagent 负责探索、提出假设和候选；类型化结构和确定性 gate 决定什么能 ship。这种分离确保安全属性，无 regression、无未审计 edit，不依赖 LLM subagent 不犯错。

4.4 适应循环

Algorithm 1 形式化了适应循环，每个 iteration 对应实验中的一轮。从初始 harness $\mathcal{H}_0$ 开始，每轮在 adaptation batch 上执行当前 harness，并选择性调用四阶段：Digester、Planner、Evolver 分别基于继续条件进行 gating，Critic 和确定性 gate 对任何到达它们的候选强制执行。只有候选通过所有 acceptance check，一轮才会提交新 harness。

4.5 通过 Ensemble Routing 做 Variant Isolation

适应循环维护单个 harness $\mathcal{H}_t$ 。当任务需要冲突行为时，一个改进某个子集的 edit 可能让另一个子集退化；seesaw constraint 会拒绝它，保护稳定性，但也丢掉了局部有益变化。Variant isolation 通过维护最多 K 个 harness variants $\{\mathcal{H}_t^{(1)}, \dots, \mathcal{H}_t^{(V_t)}\}$ 来突破这一限制，其中 $V_t \leq K$ ，并把每个任务路由到在历史轮次中对该任务 cluster 估计成功率最高的 variant。我们把这个机制称为 Ensemble routing。

Gating layer 将每个候选区分为两种结果：第一，edit 改进部分任务且不导致任何退化，则应用到目标 variant；第二，edit 改进一个子集但让另一些退化，则系统 fork 一个新 variant，而不是直接拒绝该 edit，如果 variant 池已满，则淘汰最低表现的 variant。一旦存在多个 variant，seesaw constraint 的作用域就变成 per-variant：面向 variant k 的候选只针对路由到 k 的任务测试，因此一个 cluster 的改进不会退化另一个 cluster。该设计预测了三点，并在实验中验证：聚合轨迹不退化，peak=final；更多轮次内保持探索；总 token 消耗更低。

5. Harness-Model 协同演化

前文显示，在 foundation model 固定的情况下，仅演化 harness 已经能带来显著收益，而且收益对更弱、更小的 task agent 最大，因为更好的 harness 最容易弥补它们的行为缺口。协同演化不是替代这条路径，而是在第二个轴上扩展它。对能力受限的小模型来说，harness 演化最终会遇到 scaffolding ceiling：一旦 harness 暴露了正确的工具、上下文和控制流，约束就变成冻结模型是否真的能利用它们，而没有任何 harness edit 能补足模型本身缺少的推理能力。

对称地，在固定 harness 下训练模型会遇到 training-signal ceiling：当 scaffold 从不暴露能激发新能力的上下文、工具或控制流时，新获得的能力不会被使用。模型是 agent 的 cognitive core，提供推理和规划；harness 是它的 executive apparatus，决定模型感知什么、能调用什么、执行受什么约束。更锋利的装置无法弥补虚弱的核心，更强的核心也无法弥补从不调用它的装置。协同演化正是针对这个瓶颈：在演化 harness 的同一循环中训练模型，让 agent 沿两个轴同时改进，突破单独任一路径留下的 ceiling。类似“联合演化互补能力组件”的原则也出现在其他场景中，例如 K2 Agent 协同演化层级移动设备控制中的 know-what 和 know-how。

图 3 展示了协同演化机制。HarnessX 不是在独立的 harness-evolution 和 model-training 阶段之间交替，而是在共享 replay buffer 上的单个 iteration 中同时运行二者。我们形式化该 iteration，描述两个优化底座，指定通过 cross-harness GRPO 进行模型训练的目标，并刻画共享 buffer 上的 off-policy training，这一性质使 model RL 不增加额外 rollout 成本。

图 3 : Harness-model 协同演化循环

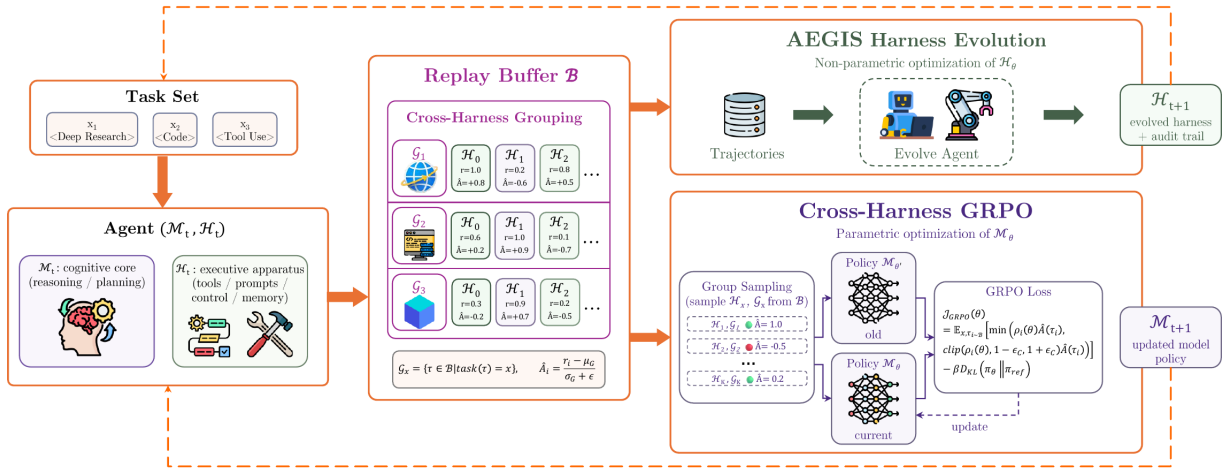


图 3 原图；图内英文已在下方表格翻译为中文。

图内文字翻译：

原文/符号	中文含义
$\mathcal{M}_t, \mathcal{H}_t$	当前模型与当前 harness
$\mathcal{B}$	共享 replay buffer
$\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3$	按任务分组的轨迹集合
$\mathcal{H}_0, \mathcal{H}_1, \mathcal{H}_2$	不同 harness version
r	reward
$\hat{A}$	group-relative advantage
$\mathcal{H}_{t+1}$	演化后的 harness
$\mathcal{M}_{t+1}$	更新后的模型
$\mathcal{J}_{GRPO}$	GRPO 目标函数

图注翻译：Agent $\mathcal{M}_t, \mathcal{H}_t$ 在固定 verifier 和 observability layer 下运行任务 batch $\mathcal{B}_t$ ；得到的轨迹和奖励 (τ, r) 进入共享 replay buffer $\mathcal{B}$ 。在 buffer 中，cross-harness grouping 将同一任务在不同 harness version 下的轨迹聚合，并计算 group-relative advantage $\hat{A}$ 。同一个 buffer 用同一批数据驱动两类更新：AEGIS harness 演化，经过 Digester $\rightarrow$ Planner $\rightarrow$ Evolver $\rightarrow$ Critic，产出演化后的 harness $\mathcal{H}_{t+1}$ ；以及 cross-harness GRPO，经过 group sampling 和 clipped GRPO objective，产出更新后的模型 $\mathcal{M}_{t+1}$ 。两者共同进入下一轮。

5.1 协同演化 iteration

协同演化作用在一对对象 $(\mathcal{M}_t, \mathcal{H}_t)$ 上，其中 $\mathcal{M}_t$ 表示可训练模型参数，放宽了前文固定模型的假设； $\mathcal{H}_t$ 表示 iteration t 的 harness 配置。系统维护一个固定容量、先进先出的 replay buffer $\mathcal{B}$ 。每个 iteration 如下：

1. Rollout。用 $(\mathcal{M}_t, \mathcal{H}_t)$ 运行 adaptation batch B_t ；observability layer 把每个 episode 记录为完整 trace τ_i ，捕获每个模型 turn、tool call 和 tool result。
2. Verification。固定 verifier 把每条 trace 评分为标量 reward r_i 。保持 verifier 固定，使不同 harness version 之间的 reward 可比较，这是 cross-harness advantage 所需要的。
3. Buffer insertion。把每条带分 trace 连同产生它的 harness version 加入共享 buffer $\mathcal{B}$ ，使后续轮次累积而不是覆盖；FIFO eviction 让 $\mathcal{B}$ 只保留近期轮次。
4. Harness evolution。
 $\mathcal{H}_{t+1} \leftarrow \text{AEGIS}(\mathcal{H}_t, \mathcal{B})$ 。Meta-agent 把 buffered traces 当作 scaffold 失效位置的证据，提出一个离散结构 edit，并且只有 Critic 和 gating layer 验证通过才接纳。
5. Behavior log-probabilities。对本轮刚加入的 traces，在生成模型 $\mathcal{M}_t$ 下跑 forward pass，得到 token-level log-probabilities $\pi_{\theta_{\text{old}}}(\tau_i)$ ，并缓存供 GRPO loss 使用；更早轮次的轨迹复用其插入时缓存的值。
6. GRPO update。 $\mathcal{M}_{t+1} \leftarrow \text{GRPO}(\mathcal{M}_t, \mathcal{B})$ 。把 traces 分成跨 harness version 的 per-task groups，为每条 trace 分配 group-relative advantage，并在固定 reference 的 KL anchor 下做 clipped policy-gradient step。
7. Advance。用演化后的 pair $(\mathcal{M}_{t+1}, \mathcal{H}_{t+1})$ 回到步骤 1。

每条 trace 同时作为 AEGIS 的诊断证据和 GRPO 的训练信号。Harness 演化和模型更新读取同一个 buffer，但在同一 iteration 内互不依赖对方输出；二者都完成后才进入下一次 rollout。

5.2 优化底座

Harness 侧：非参数优化。Harness 演化按前文 AEGIS 运行，从 replay buffer $\mathcal{B}$ 中读取 trace evidence。与 standalone AEGIS 的主要区别是， $\mathcal{B}$ 包含来自多个模型 checkpoint 的轨迹， $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_t$ ，使 Digester 能看见来自模型更新和 harness edit 两方面的行为变化。

模型侧：通过 GRPO 做参数优化。关键设计选择是 cross-harness grouping criterion：所有共享同一 task identifier 的轨迹组成一个 GRPO group，不管它们由哪个 harness 或模型 checkpoint 产生。因此，组内变化反映的是策略差异，而不只是采样噪声。

互补性。Harness update 做离散结构变化，例如添加工具、替换 control processor、重构 prompt，这些无法表示为参数更新。Model update 做细粒度行为调整，例如何时调用哪个工具、如何 phrasing query、何时停止，这些依赖高维 in-context state，也无法用符号规格捕获。Harness 定义粗粒度策略架构；模型学习利用它。

5.3 通过 Cross-Harness GRPO 训练模型

本文采用 Group Relative Policy Optimization。形式化地，buffer 中每条 trajectory 由下式生成：

$$\tau_i \sim \text{Agent}(\mathcal{M}_k, \mathcal{H}_{k,x_i}), \quad k \in \{0, 1, \dots, t\}$$

其中 i 是 buffer $\mathcal{B}$ 中的 (x, τ) 索引， $\mathcal{M}_k$ 和 $\mathcal{H}_k$ 是把任务 x_i rollout 为轨迹 τ_i 的模型 checkpoint 与 harness。由于 FIFO eviction 把 buffer 限制在近期 iteration，buffered trajectories 来自接近当前 policy 的模型 version。但它们在策略上差异显著，例如 tool selection、prompt structure、control-flow logic，这些差异来自 successive harness versions。不同于 single-policy RL 中组内变化主要来自随机采样，这里 harness identity 主导变化，因此 cross-harness grouping criterion 对有意义的 advantage estimation 至关重要。

对于任务 x ，trajectory group 收集所有 x 的 trace，不管它们由哪个 $(\mathcal{M}_k, \mathcal{H}_k)$ pair 产生：

$$\mathcal{G}_x = \{\tau_i \in \mathcal{B} \mid \text{task}(\tau_i) = x\} \\ = \bigcup_k \{\tau \sim \text{Agent}(\mathcal{M}_k, \mathcal{H}_{k,x})\}$$

因此，模型接收的是跨策略 reward 对比产生的梯度信号，而不是固定策略内随机变化产生的信号，这使它能够内化跨 harness version 成功的策略。

Task-level alignment，而非 action-level alignment。Cross-harness GRPO 做的是 task-level alignment：来自不同 harness version 的 trajectories 按 task identity 分组，只通过 verifier reward 比较。不需要 action-level alignment，因此 action space 不兼容的 harness version，也就是不同 tool schema、不同 prompt structure、不同 control-flow processor，可以共存于同一组而不冲突。计算 policy gradient 时，每条 trajectory τ_i 都在产生它的 harness version $\mathcal{H}_k$ 下 replay：模型 log-probabilities $\pi_\theta(\tau_i \mid x)$ 是相对于 $\mathcal{H}_k$ 在每个 turn 构造的 prompt、tool schema 和 observation context 评估的。因此 GRPO 梯度完全作用于以 harness-specific context 为条件的模型输出 token，而不是作用于 harness structural action 或 environment transition。这一设计把 harness evolution 和 model training 解耦：harness evolution 可以自由改变跨 version 的 action space；model training 只要求每条 trajectory 自己 harness context 下的 token-level log-probabilities。

Group-relative advantage 为：

$$\hat{A}(\tau_i) = \frac{r_i - \mu(\mathcal{G}_x)}{\sigma(\mathcal{G}_x) + \epsilon}$$

其中 r_i 是 trajectory τ_i 的 reward， $\mu(\mathcal{G}_x)$ 和 $\sigma(\mathcal{G}_x)$ 是组内 reward 均值和标准差。演化中的 harness 作为模型 RL 的结构化探索算子：每个新版本给任务采样分布注入一种不同的行为模式，advantage 会把模型推向 verifier 评分最高的模式。单策略采样无法提供的探索广度，由演化 scaffold 自身提供。

最大化的 policy objective 为：

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{x, \tau \sim \mathcal{B}} \left[\min_{\left(\rho_i(\theta) \hat{A}(\tau_i), \text{clip}(\rho_i(\theta), 1 - \epsilon_c, 1 + \epsilon_c) \hat{A}(\tau_i) \right)} \right] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

其中：

$$\rho_i(\theta) = \frac{\pi_{\theta}(\tau_i \mid x)}{\pi_{\theta_{\text{old}}}(\tau_i \mid x)}, \quad \pi_{\theta_{\text{old}}} = \mathcal{M}_d$$

这是当前 policy $\mathcal{M}_k$ 与生成 τ_i 的 checkpoint $\mathcal{M}_d$ 之间的 importance-sampling ratio； ϵ_c 是 clipping threshold； $\beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ 惩罚相对固定 reference model π_{ref} 的偏离。Ratio 中的 behavior policy $\pi_{\theta_{\text{old}}}$ 和 KL 项中的 reference policy π_{ref} 是不同的： $\pi_{\text{ref}} = \mathcal{M}_0$ 在训练中固定，而 $\pi_{\theta_{\text{old}}}$ 随 trajectory 变化，必须从 buffer 中恢复。

5.4 混合策略 Buffer 上的 Off-Policy Training

Replay buffer 本质上是 off-policy 的：iteration t 时，它持有由 checkpoint $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_t$ 在 harness $\mathcal{H}_0, \mathcal{H}_1, \dots, \mathcal{H}_t$ 下生成的 trajectories，因此 buffer distribution 不匹配正在更新的 policy π_{θ} 。为每条 buffered trajectory 恢复 $\pi_{\theta_{\text{old}}}$ 是核心挑战。

Behavior policy $\pi_{\theta_{\text{old}}}$ 。Importance ratio 修正 π_{θ} 和生成 τ_i 的 checkpoint $\mathcal{M}_k$ 之间的差距。由于 $\mathcal{M}_k$ 在 buffer 中变化， $\pi_{\theta_{\text{old}}}(\tau_i)$ 不能从单一模型恢复：我们在 buffer insertion 时，用 $\mathcal{M}_k$ 跑一次 forward pass，把 token-level log-probabilities 物化并缓存到磁盘，然后在每次 gradient step 重用。这把 cached behavior log-probabilities 和每步重新计算的 current log-probabilities $\pi_{\theta}(\tau_i)$ 解耦。

有界 off-policy bias。FIFO eviction 把 buffer 限制在 C 条 trajectory；若每轮有 s 个样本，则最大 model-version lag 为 $\lfloor C/s \rfloor$ 轮，因此每个 cached $\pi_{\theta_{\text{old}}}$ 都来自 π_{θ} 的有限窗口，生成 trajectory 的 policy 不会与正在更新的 policy 相差太远。同一个窗口也限制 harness staleness，因此 cross-harness groups 只混合近期 scaffold version，模型不会主要针对过时 harness 训练。

不增加 rollout 成本的 replay reuse。Agentic RL 的主成本是 rollout，即在环境中执行 agent：模型解码、工具调用、验证，而不是 gradient update。协同演化中，单轮探索产生一组 trajectories，服务两种更新：同一批 traces 驱动 AEGIS harness update，也通过共享 buffer 驱动 cross-harness GRPO model update。GRPO 通过 replay 消费这些 trajectories，不产生自己的 rollout。因此，添加 model update 的边际成本仅限于：每条 trajectory 一次 cached forward pass，用来记录 $\pi_{\theta_{\text{old}}}$ ；以及 gradient steps 本身。二者都不需要 rollout。没有任何 trajectory 是专门为了训练模型而生成的。因此联合优化是经济的：它用离线训练

compute 的价格换取模型改进，不需要超出 harness evolution 已经执行的 rollout。

6. 实验

我们沿五个轴评估 HarnessX：跨 benchmark 和模型家族的总体效果；variant-management strategy 对稳定性的影响；evolver 架构相对于基础设施的贡献；model-harness 协同演化的收益；以及对预测失败模式的经验证实。

6.1 实验设置

Benchmarks。如表 3 所示，我们在五个 benchmark 上评估，覆盖多步检索、具身规划、Web 交互、多轮对话和软件工程。除非另有说明，每个实验最多运行 $T=15$ 轮演化，连续 $P=3$ 轮没有 shipped edit 时提前停止。每轮评估完整任务集，不做 subsampling。Meta-agent token budget 随 benchmark 变化，总量 100M-175M，但同一 benchmark 内不同 task agent 保持一致。

表 3：Benchmark 特征

Benchmark	Domain	Sampled Tasks	Verifier
GAIA (Level 1-3)	多步检索	103	Exact match
ALFWorld	具身规划	134	Goal completion
WebShop	Web 交互	100	Attribute match
τ^3 -Bench	多轮对话	3 domains	Rule compliance
SWE-bench Verified	软件工程	55	Patch resolution

Models。我们区分两个角色：meta-agent，默认为 Claude Opus 4.6，驱动 AEGIS 演化循环；task agent，在演化后的 harness 下解决 benchmark 任务。Task agents 覆盖三个家族：Claude Sonnet 4.6、GPT-5.4、Qwen3.5-9B，用于测试单个 meta-agent 是否能跨模型家族演化有效 harness。

Baselines。

1. Static Harness：由已发布 benchmark-specific prompt 和 tool definition 构造的 HarnessX 配置，在所有轮次中固定。
2. Claude Code SDK (CC SDK)：单 agent evolver，每轮一个 LLM session，替代四阶段 pipeline，同时保留相同基础设施和轮次预算，从而隔离 AEGIS 多阶段架构相对共享基础设施的影响。该 baseline 也作为 SICA 等 monolithic evolver 的 proxy。实验使用 Claude Code SDK v0.0.25，model="opus"，Claude Opus 4.6，max_turns=200，实验时间为 2026 年 5 月。

Metrics。任务成功率，按 benchmark-specific verifier 计算。每个任务每轮有两次独立 attempt，pass@2 表示任一 attempt 成功即视为解决；这降低采样噪声，同时保留 seesaw constraint 所需的二值 per-task 信号，代价是会掩盖低于阈值的成功概率漂移。

Scope。所有报告增益都在用于演化的同一任务集上测量；本文不评估对未见任务的 held-out generalization。

图 4：演化轨迹

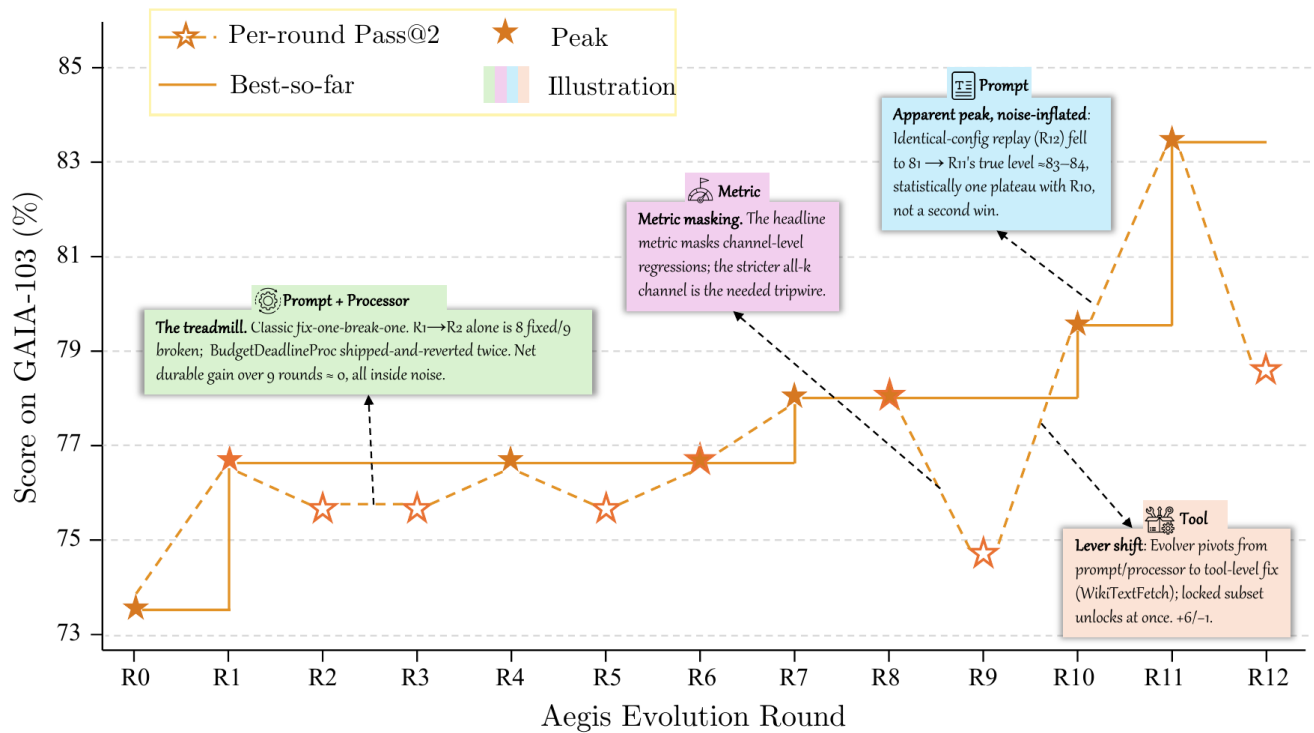


图 4 原图；图内英文已在下方表格翻译为中文。

图注翻译：演化轨迹，纵轴为 pass@2 success rate，横轴为 round。虚线是 static-harness baseline。

图内文字翻译：

原文	中文
Per-round Pass@2	每轮 pass@2
Best-so-far	历史最佳
Peak	峰值
Aegis Evolution Round	AEGIS 演化轮次
Score on GAIA-103 (%)	GAIA-103 分数
Prompt / Processor / Metric / Tool	提示词 / Processor / 指标 / 工具
The treadmill. Classic fix-one-break-one.	跑步机效应：典型的修一个坏一个。
Metric masking. The headline metric masks channel-level regressions.	指标遮蔽：headline 指标遮住了 channel-level regression。
Lever shift: Evolver pivots from prompt/processor to tool-level fix.	杠杆转移：Evolver 从 prompt/processor 转向 tool-level 修复。

6.2 主要结果

表 4 和图 4 报告 harness 演化前后的 pass@2 success rate。AEGIS 改进了 15 个 model-benchmark 配置中的 14 个，平均提升 +14.5%，最高 +44.0%。唯一停滞的配置是 GAIA / GPT-5.4, $\Delta=0.0$ ，反映了单 harness 演化在异质任务集上的基本限制；variant isolation 能解决这一点。一个配置在中途退化， τ^3 -Bench Telecom 于 R7 下降 -14.0%，原因是累积同类型 edit，并在 R9 恢复。

表 4：主要结果

Benchmark	Task agent	Initial	Evolved	Δ	Best round
ALFWorld	Claude Sonnet 4.6	83.6	94.8	+11.2	7
ALFWorld	GPT-5.4	76.9	97.8	+20.9	4
ALFWorld	Qwen3.5-9B	53.0	97.0	+44.0	9
WebShop	Claude Sonnet 4.6	60.0	76.0	+16.0	7
WebShop	GPT-5.4	55.0	73.0	+18.0	8
WebShop	Qwen3.5-9B	36.0	49.0	+13.0	7
GAIA	Claude Sonnet 4.6	73.8	83.5	+9.7	11
GAIA	GPT-5.4	73.8	73.8	0.0	4
GAIA	Qwen3.5-9B	20.3	37.4	+17.1	4
SWE-bench Verified	Claude Sonnet 4.6	76.4	87.3	+10.9	3
SWE-bench Verified	GPT-5.4	45.5	63.6	+18.2	3
SWE-bench Verified	Qwen3.5-9B	23.6	41.8	+18.2	2
τ^3 -Bench (Avg.)	Claude Sonnet 4.6	89.6	95.0	+5.4	--
τ^3 -Bench (Avg.)	GPT-5.4	76.2	90.7	+14.5	--
τ^3 -Bench (Avg.)	Qwen3.5-9B	93.5	94.6	+1.1	--

总体表现。演化改进了 15 个配置中的 14 个。ALFWorld 的提升范围为 +11.2% 到 +44.0%，WebShop 为 +13.0% 到 +18.0%，SWE-bench Verified 为 +10.9% 到 +18.2%。在 GAIA 上，Sonnet 4.6 和 Qwen3.5-9B 分别提升 +9.7% 和 +17.1%，而 GPT-5.4 停滞， $\Delta=0.0$ ；解决其失败需要互相冲突的 edit，单 harness strategy 无法容纳。在 τ^3 -Bench 上，GPT-5.4 提升最大，+14.5%；Qwen3.5-9B 只提升 +1.1%，因为其 93.5% 基线已接近天花板。

与基线性能反向 scaling。跨 benchmark 看，最弱 task agent Qwen3.5-9B 一贯收益最大：ALFWorld +44.0%，基线 53.0%；GAIA +17.1%，基线 20.3%；SWE-bench Verified +18.2%，基线 23.6%。更强模型在 ALFWorld 和 SWE-bench 上收益更小。例外是 GAIA GPT-5.4， $\Delta=0.0$ ，其任务异质性阻止单个 harness 提升 aggregate accuracy，也推动了 variant-isolation 消融。整体模式表明，弱模型表现出更多可由 harness-level edit 处理的行为缺口；当基线表现足够高时，剩余失败越来越需要 task-specific adaptation，而非全局改进。

跨模型泛化。Meta-agent Opus 4.6 能为不同模型家族的 task agent 演化 harness，而不需要 family-specific adaptation。ALFWorld 上，跨家族 agent，GPT-5.4 +20.9%，Qwen3.5-9B +44.0%，比同家族 agent Sonnet 4.6 +11.2% 获益更大，说明增益大小跟基线表现相关，而不是跟 meta-agent 家族接近程度相关。

收敛速度跟随 failure-mode 集中度。ALFWorld 的 GPT-5.4 在 R4 达峰，SWE-bench Verified 的所有 agent 在 R2-R3 达峰；两者失败都集中在一两种 component type，因此快速收敛。GAIA 的 Sonnet 4.6 需要 11 轮，因为失败横跨 prompt、tool、processor、configuration 四种 component type，需要顺序探索多个 edit neighborhood。

τ^3 -Bench 内部的 domain-level variation。平均 τ^3 -Bench 增益掩盖了显著的 per-domain variation。GPT-5.4 在 Telecom 上从 67.5% 提升到 93.0%，R2 +25.4%；在 Retail 上从 84.2% 到 93.9%，R6 +9.7%。然而，Sonnet 4.6 在 Telecom 上因为累积同类型 edit，于 R7 单轮退化 -14.0%，并在 R9 恢复。这展示了 per-edit gating 的结构性限制：连续同类型 edit 带来的低于阈值 coupling 会累积，直到 tipping point 触发可见 regression。

SWE-bench 上的 post-peak degradation。SWE-bench Verified 的 GPT-5.4 在 R3 达到 63.6%，+18.2%，但到 R5 退化到 50.9%，相对峰值下降 -12.7%；最终 accuracy 仍比 static baseline 高 +5.4%。两个因素加速了这个 benchmark 上的退化：第一，只有 55 个任务，每个 task flip 会让 aggregate accuracy 变化约 1.8%，而 n=103 时约 1.0%；第二，结构性代码 edit 的 blast radius 比 prompt edit 更广。这与 GAIA GPT-5.4 停滞类似：两者都推动了 variant-isolation strategy。

6.3 演化策略比较

主实验使用 Global strategy：在所有任务上演化单个 harness。表 5 在 GAIA 上比较了默认 Global 和 variant-isolation strategy，103 个任务，GPT-5.4，15 轮，AEGIS evolver。

表 5：演化策略比较

Strategy	Final (%)	Peak (%)	Final-Peak	Tokens
Ensemble (up to K variants)	87.4	87.4	0.0	107.8M
Global (single harness)	49.5	73.8	-24.3	143.7M

Global 的失败机制。Global strategy 为全部 103 个任务维护单个 harness。它在 R4 早早达到 73.8% 的峰值，随后持续退化：后续 edit 引入了低于阈值的 regression，在 pass@2 的二值信号下单独不可见，却会累积成 aggregate decline。peak-final gap 为 -24.3%，远超每轮 binomial 95% confidence interval，约 $n=103, p \approx 0.74$ 时的 $\pm 8.5\%$ ，因此排除评估噪声并确认 catastrophic forgetting。这解释了 GAIA GPT-5.4 在表 4 中 $\Delta=0.0$ 的停滞：Global 不能在异质任务集上持续改进。

Ensemble 为什么防止 cross-variant forgetting。Ensemble routing 维护最多 K 个 harness variants，并把每个任务路由到历史成功率最高的 variant。Edit 按 variant 提出和评估，因此改进一个 cluster 的 edit 不会让另一个 cluster 退化。对比验证了三种预测性质：聚合轨迹不退化，peak=final；更晚达峰，R14 对 R4，说明保持了持续有效探索；token 消耗更低，107.8M 对 143.7M，因为每个 edit 只针对目标 cluster 评估，而不是全任务集，也避免了 degraded single harness 在全任务集上浪费 proposal。

小结。Variant isolation 解决了 Global 下的停滞，把 GAIA GPT-5.4 从 $\Delta=0.0$ 提升到 +13.6%，87.4%，且不退化。更细粒度策略，如 Domain-aware clustering 和 Task-level tournament，只做了 pilot scale，30-40 个任务、最多 8 轮，缺乏足够轮次和任务进行统计比较。

6.4 Meta-Agent 有效性

为分离 evolver 架构和基础设施，我们用单 agent CC SDK evolver 替代四阶段 AEGIS pipeline。它使用相同模型 Opus 4.6、相同轮次预算和基础设施。两个 evolver 都在 variant isolation 下运行，以确保 non-degrading trajectory。表 6 报告了 GAIA 上的对比，103 个任务，GPT-5.4，15 轮。

表 6：Meta-agent 架构比较

Evolver	Accuracy (%)	Best round	Tokens
AEGIS	87.4	R14	107.8M
CC SDK	86.4	R12	123.1M

Accuracy 相近；efficiency 不同。1.0% 的 accuracy 差距落在一个 standard error 内， $n=103$ 时约 3.3%，说明在这个 meta-agent 能力水平下，四阶段分解没有提高最终 accuracy。然而，单 agent 变体消耗约 14% 更多 token，123.1M 对 107.8M。我们把这归因于 Digester 压缩：它把约 10M raw trace tokens 压缩到约 10K structured summaries，再供下游阶段消费。没有这一阶段，单 agent evolver 必须截断 traces 以适应 context window，产生信息更少的 edit，更常被 gate 拒绝，从而浪费 token。

含义。在 capable meta-agent 和 variant isolation 下，accuracy gain 主要来自 HarnessX 的基础设施，类型化组件使 isolation 成为可能，结构化 traces 使诊断成为可能，而不是来自 evolver 的内部架构。四阶段分解贡献的是效率，约少 12% token，以及可解释性，可审计的中间 artifact，但在这个规模上没有可测的 accuracy 改进。

6.5 协同演化

该实验测试把 harness evolution 和 model RL 交错运行，是否能超过 harness-only evolution。如图 5 所示，我们在 GAIA 和 WebShop 上用 Qwen3.5-9B task agent 比较两种 regime。两个条件共享固定容量 FIFO replay buffer：每轮让当前 agent 在 adaptation batch 上运行，固定 verifier 给得到的 traces 打分，harness evolution 和 model training 都在同一 buffer 上更新。第 5 节预测，单一优化路径都会卡在各自 ceiling：harness-only 卡在 scaffolding ceiling，model-RL-only 卡在 training-signal ceiling。协同演化通过让模型内化 successive harness versions 引入的策略，处理两个 ceiling。

实验设置。两种 regime 都在 GAIA text-only subset，103 个任务，和 WebShop subset，100 个任务上运行，task agent 为 Qwen3.5-9B。GAIA 使用实时 web tools，其延迟和可用性会波动，因此每轮评估两次并取平均。两个 subset 都很小，所以 optimizer batch 设为整个 replay buffer，buffer 设为四轮滑动窗口：GAIA 每个任务两个 rollout，即 824 traces，WebShop 为 400 traces，这为 GRPO 稳定估计 advantage 提供足够组内样本。训练使用 learning rate 1×10^{-6} ，GRPO clip $\epsilon=0.2$ ，无 KL penalty，coefficient 0，每轮训练 5 steps。GAIA agent 配备 web search、Baidu API、web fetch、bash、file read；WebShop 使用其环境内置 action tools。GAIA reward 为 $0.9 \times \text{correctness}$ 加 $0.1 \times \text{format}$ ；WebShop 使用原生 attribute-match reward，reward=1.0 才算 task passes。

图 5：协同演化 vs. harness-only

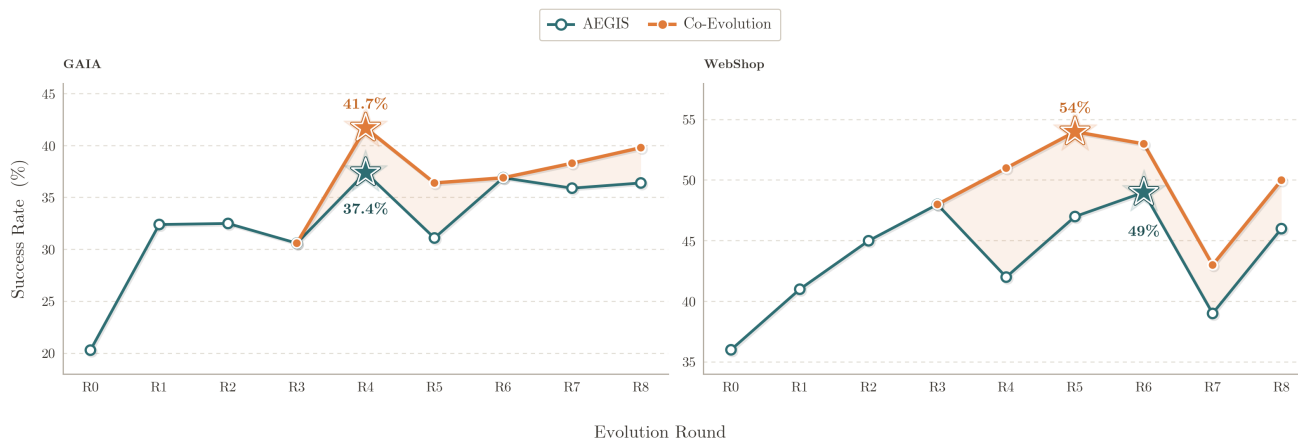


图 5 原图；图内英文已在下方表格翻译为中文。

图注翻译：GAIA 和 WebShop 上的协同演化与 harness-only 演化对比，后者使用 AEGIS 且冻结模型。星号标出每种方法的峰值；阴影带表示协同演化增益。

协同演化超过 harness-only 演化。如图 5 所示，在共享 replay buffer 上交错运行 cross-harness GRPO 和 harness evolution，使两个 benchmark 的 peak success 都提高：GAIA 从 37.4% 到 41.7%，+4.3%；WebShop 从 49.0% 到 54.0%，+5.0%；相对 model-frozen baseline 平均 +4.7%。两条曲线在 joint training 生效前，即 R4 前基本重合，随后分离；余下 run 中协同演化始终不低于 harness-only。该差距持续到最终轮，GAIA 从 36.4% 到 39.8%，WebShop 从 46.0% 到 50.0%，且 WebShop 差距更大，因为在 harness-only plateau 之外仍有更多 model-level improvement 空间。协同演化提升的是 run 结束时 accuracy，而不只是峰值。

协同演化突破 scaffolding ceiling。Harness-only evolution 在 GAIA 约 37% 和 WebShop 约 49% 处 plateau。协同演化越过这些 plateau：cross-harness GRPO 让模型内化 successive harness versions 的策略，因此后续 edit 构建在已学习行为上，而不只是补偿固定模型的内在限制。

6.6 失败分析

本文给出三个 case study，分别对应操作镜像预测的三个病理：reward hacking、catastrophic forgetting、under-exploration。每个 case 都记录第一次暴露问题的 detection signal、通过 trace analysis 找到的 root cause，以及 outcome，也就是 pipeline 是否自我纠正或需要人工干预。图 6 给出了按病理类型组织的全部 confirmed 和 pending case。

图 6：按病理组织的失败案例



图 6 原图；图内英文已在下方表格翻译为中文。

图注翻译：失败案例按病理组织，行分别为 reward hacking、catastrophic forgetting 和 under-exploration。

图内主要文字翻译见本文末尾“图表文字译表”。

Reward hacking (GAIA, Sonnet 4.6, R10)。R10 时, pipeline ship 了一个 composite edit, 覆盖 tool、prompt、configuration, manifest 预测会改进 retrieval。该 edit 通过 seesaw constraint, 并把 accuracy 从 74.8% 提到 79.6%。R11 的 trace analysis 显示, 该工具确实为多数新通过任务修复了 retrieval, 但有一部分任务是利用 verifier 的格式规律而通过, 并非实际检索。Planner 在 R12 标记了该路径, 随后 edit 引入 guard, 把工具限制在输出能被第二条 retrieval path 交叉检查的任务上。

Catastrophic forgetting (τ^3 -Bench, Sonnet 4.6, Telecom, R7)。Telecom 上的演化在 R2-R6 连续五轮 ship 同类型 prompt/processor edit, 每次都追加一个“reminder” rule。Compliance 从 89.5% 提升到 R4 的 100%, 随后到 R6 退化到 94.7%, 因为后续规则与早期规则冲突。R7 Critic 标记了集中风险, “All 5 prior ships occupy the same bucket: [prompt, processor]”, 但仍批准 ship, 因为 ship-prediction accuracy 仍高, 且没有记录 regressions。第六条 reminder 通过跨规则冲突让 compliance 从 94.7% 降到 80.7%, -14.0%, 使先前通过任务不稳定。该 regression 逃过 seesaw constraint, 因为 pass@2 只记录 per-task binary flip, 不记录低于阈值的 coupling。R9 时 pipeline 自我纠正: Planner 诊断集中模式, 并提出结构性 edit, 替换冲突的 reminder stack。

Under-exploration (ALFWorld, Sonnet 4.6, R4-R7)。R4 到 R7 之间, pipeline 主要 ship prompt-level edit, 每轮增益低于 1%。Ship-prediction accuracy, 从 manifest 预测 task flip 到实际发生的比例, 从 R3 的 80% 降到 R7 的 0%, 说明 prompt-space 已耗尽。该窗口里唯一结构性 edit 是 R6 的 processor-level change, 但 ship-prediction accuracy 只有 14%, 预测 7 个 flip 仅 1 个实现, 说明 Planner 缺少足够的 structural-edit history 来校准 prompt neighborhood 之外的假设。

小结。操作镜像预测的三个病理都在实践中出现。Pipeline 在 R10-R12 两轮内检测并缓解 reward hacking。衰减的 ship-prediction accuracy 诊断了 R4-R7 的 under-exploration。Catastrophic forgetting case 暴露 per-edit gating 的结构性限制: 低于阈值的 coupling 会累积, 直到超过 per-task 检测阈值。 τ^3 -Bench Telecom 上 pipeline 能自我纠正, 因为失败局限在一个 domain; GAIA GPT-5.4 上同一机制导致持续停滞, $\Delta=0.0$, 因为冲突 edit 阻止净增益。Variant isolation 通过把 edit 限定到 task-specific cluster 来解决这一点。

7. 讨论

7.1 为什么组合结构对演化重要

如表 5 所示, Global strategy 在 GAIA 上 R4 早早达到 73.8% 峰值, 随后跌到 49.5%, peak-final gap 为 -24.3%。Global 使用 HarnessX 的类型化组件, 但没有利用它们做 isolation; 每个 edit

都对所有任务一起评估。在 pass@2

下, 一个成功概率已经下降的任务仍可能登记为“已解”, 因此低于阈值的 regression 会逃过 seesaw constraint。防止这种崩塌需要 variant isolation, 而组合性使它成为可能: HarnessX 的组合结构让每个 edit 的预期作用范围显式化, 这是把每个 edit 的评估限制到目标 cluster、而不是不加区分地对全任务集评估的前提。

这种关系类似类型系统：类型不会自动生成正确程序，但会让错误程序可检测。类似地，类型化组件不会阻止坏 edit，但会让其作用范围显式化，从而允许独立变体。策略比较说明，稳定演化需要 variant isolation；没有组合结构，edit 的预期作用范围未定义，variant isolation 也就没有良好定义。不过，组合结构并不保证行为影响有界： τ^3 -Bench Telecom 失败显示，累积同类型 edit 会引发低于阈值的 coupling，并同时退化多个 dialogue pattern。

7.2 Trace 丰富度的作用

HarnessX 的完整执行 trace τ 提供了超越标量 reward 的诊断信息。Case study 证实了这一点：检测 GAIA 上 R10 ship、R11 检出的 reward hacking，需要检查 improvement 如何发生，是格式利用还是真实检索；检测 ALFWorld 上 R4-R7 的 under-exploration，需要跟踪 edit-type distribution 和 ship-prediction accuracy。这些信号都无法从 per-task binary outcome 中恢复。

这些观察推动了一个设计原则：反馈信号的丰富度，限制了能安全执行的演化复杂度。仅凭标量 reward，三个病理都不可检测：分数变化不能区分 reward hacking 和真实改进，不能区分 under-exploration 和收敛，也不能区分 catastrophic forgetting 和评估噪声。Trace structure 使每种病理可诊断，前提是存在 prior-round traces 用于对比。 τ^3 -Bench Telecom 失败说明了边界：尽管有 R2-R6 五轮 prior traces，累积 regression 仍逃过 seesaw constraint，因为没有单个 edit 超过检测阈值。因此，结构化 trace 记录是检测病理的必要条件，但不足以防止病理：当 coupling 在 per-task 检测阈值以下累积时，trace 只能在损害发生后记录症状。

7.3 操作镜像的范围与限制

RL-symbolic-space mirror 是设计启发式，不是形式框架。经典 RL 收敛保证需要充分探索 state-action space，而当 state 是符号 harness configuration、action 是开放式代码 edit 时，这个条件无法满足。在 Global strategy 下，GAIA GPT-5.4 完全停滞，15 轮 $\Delta=0.0$ ；variant-isolation 消融恢复稳定提升，最终 87.4% 且 peak=final，但没有保证它能延伸到更长 horizon，因为 variant 可能 over-specialize，也没有保证能处理 inter-task dependencies 阻止干净 variant separation 的任务分布。镜像也不能预测哪个病理会主导： τ^3 -Bench Telecom 上 catastrophic forgetting 在 R7 浮现；ALFWorld 上 under-exploration 主导 R4-R7；GAIA 上 reward hacking 直到 R10 才出现。

因此，我们把镜像当作设计 checklist，而不是预测理论：它识别需要防御的失败模式，但不预测它们的排序、时机或相对严重程度。三个病理只是代表性而非穷尽；其他 RL 现象，例如 adaptation batch 与 deployment tasks 不一致时的 distribution shift，或 hard benchmark 上的 reward sparsity，也可能在符号空间中表现为类似故障模式。

7.4 跨模型家族泛化

在 ALFWorld 上，Opus 4.6 meta-agent 为三个模型家族的 task agent 演化 harness：

- Sonnet 4.6，同家族：83.6% $\rightarrow$ 94.8%，+11.2%。

- GPT-5.4，不同家族：76.9% -> 97.8%，+20.9%。
- Qwen3.5-9B，不同家族且更弱：53.0% -> 97.0%，+44.0%。

Inverse-scaling effect 解释了幅度排序：增益跟基线表现反向相关，Qwen > GPT > Sonnet，而不是跟 meta-agent 模型家族接近度相关。三个配置都固定 meta-agent 为 Opus 4.6，只改变 task agent；本文没有评估更弱 meta-agent 是否能获得类似收益。

补充消融发现，在共享相同 meta-agent 模型和基础设施时，单 agent evolver 和四阶段 AEGIS pipeline 的 accuracy 相当，86.4% 对 87.4%，差异在采样噪声内。这说明，在这个 meta-agent 能力水平下，四阶段分解主要提供效率收益，约少 12% token，以及可审计性，而不是可测的 accuracy 改进。

7.5 成本-性能权衡

如表 7 所示，演化产生前置 compute 成本，但可在后续任务调用中摊销。

表 7：演化成本摘要

Experiment	Rounds	Total Tokens	Gain
GAIA, GPT-5.4 (Global)	15	143.7M	0.0%，peak=initial
GAIA, GPT-5.4 (Variant isolation, ablation)	15	107.8M	+13.6%
ALFWorld, Sonnet 4.6 (Global)	7	43.4M	+11.2%

Strategy ablation 显示，在 GAIA 上，variant isolation 比 Global 更有效，final 87.4% 对 49.5%，也更高效，107.8M 对 143.7M tokens。Token 减少有两个来源：结构上，variant isolation 下每个 edit 只针对目标 cluster 评估，而不是全任务集，从而降低 per-round evaluation cost；Global 下持续退化的 baseline 导致更多候选 gate 失败，浪费 token 在不会 ship 的候选上。在快速收敛的 benchmark 上，例如 ALFWorld R4-R7、SWE-bench R2-R3，Global 足够使用，退化没有在 run horizon 内出现。

演化后的 harness 也影响 per-task inference cost。GAIA 上，per-task token consumption 下降约 25%，因为 targeted tool selection 缩短轨迹；ALFWorld 上则上升约 60%，因为 task-decomposition prompt 拉长执行。

部署时，演化后的 harness 是静态 artifact，不需要 meta-agent inference；演化集之外的任务被路由到在演化集上总体成功率最高的 variant。GAIA 上，107.8M tokens 的前置成本可在约 1,300 次调用内摊销，每次约省 83K tokens。ALFWorld 上 per-task cost 增加，回报是 accuracy，+11.2%，而不是 cost reduction。

7.6 伦理考虑

自演化 agent 系统需要显式 oversight。HarnessX 提供三种机制：

1. 可审计性：每个 shipped edit 都携带 manifest 和 rollback target；rejected candidates 以拒绝原因归档。
2. 确定性 gating：seesaw constraint 会拒绝任何使先前已解任务在 pass@2 下退化的 edit。
3. Human-in-the-loop：gating layer 支持对超过可配置风险阈值的 edit 进行人工批准；本文自动化实验中未使用。

τ^3 -Bench 失败说明了这些机制的限制：R2-R6 连续五个同类型 edit 累积了低于阈值的 coupling，seesaw constraint 未检测到；第六个 edit 在 R7 触发可见 -14.0% regression，但没有单个 edit 违反约束。这是 per-edit gating 的结构限制：低于阈值的 regression 会累积，即使先前多轮在同一约束下看似稳定。

7.7 局限

除上述限制外，还有五个约束限制本文结果的普遍性：

- 没有 held-out evaluation。所有报告增益都在用于演化的同一任务集上测量。由于我们报告 peak accuracy，并在 adaptation set 本身上评估，数字包含 selection bias 和潜在 overfitting。对同分布未见任务的泛化是可能的，但未测试。
- 只有离散 action space。所有实验使用具有离散文本 action space 的 agent。尚未测试框架是否能扩展到连续 action space，例如机器人控制。
- 闭源 meta-agent。AEGIS 需要能做多文件代码生成、结构化 trace analysis 和多步规划的 meta-agent。接近这一能力水平的 open-weight 模型，例如 Qwen3.5-72B、Llama-4-Maverick，尚未作为 meta-agent 测试。
- Joint control assumption。协同演化需要同时控制 harness evolution 和 model training。实践中，这两个关注点常由不同团队或组织负责，没有跨团队协调时，共享 replay buffer 不现实。
- Benchmark 覆盖。所有 SWE-bench Verified 运行都使用 55-task subsample， τ^3 -Bench 只评估 Retail、Airline、Telecom 三个 domain。结论，尤其 inverse-scaling effect，可能无法泛化到具有不同任务异质性的 domain 或更大评估集。

8. 结论

本文提出 HarnessX，一个可组合的运行时代 foundry，把 harness 视为模型和环境之间的一等接口。这个接口可以由类型化 primitive 组合，可以从执行轨迹中演化，也可以在统一改进循环中与模型训练耦合。跨五个 benchmark 和三个模型家族，HarnessX 通过组合底座上的轨迹驱动演化，实现最高 +44.0% 的提升，在 15 个配置上平均 +14.5%；协同演化在两个 benchmark 上比 harness-only evolution 额外增加 +4.7%。这些结果说明，agent 进步不必只依赖模型 scaling：从执行反馈中组合并演化运行时接口，是一条互补且可操作的杠杆，尤其适用于能力受限 agent，因为它们的 harness-level gain 最大。

参考文献

参考文献保持原始英文书目信息，见同目录

references-original.bib。这样做不是省事，是为了不破坏标题、作者名、URL 和 citation key 的可检索性。

贡献与致谢

Core Contributors :

- Tingyang Chen*
- Shuo Lu*
- Kang Zhao*
- Weicheng Meng
- Kun Shao†
- Jian Luan†

Contributors :

- Hanlin Teng
- Tianhao Li
- Chao Li
- Xule Liu
- Jian Liang
- Zhizhong Zhang
- Yuan Xie
- Heng Qu

* Equal Contribution ; † Corresponding Author 。

附录

A. 实验设置：完整细节

本附录扩展 6.1 节中的简要设置，包含完整 benchmark 描述、正式 metric definition、演化协议超参数和运行时基础设施。

A.1 Benchmarks

我们选择五个 benchmark，用来覆盖 harness design 最影响的失败模式，从短周期具身规划到长周期软件工程。

GAIA。GAIA benchmark 提出对人类概念上简单，但需要 agent 组合多个 action 的真实世界问题，例如 web search、file extraction、multimodal interpretation、arithmetic，并通过 exact match 和参考答案比较。该 benchmark 强调开放式 tool-based reasoning，其中 harness 决定证据如何收集和综合。

ALFWorld。ALFWorld benchmark 涉及 embodied instruction following：文本 agent 在家庭场景中命令模拟机器人 agent。给定自然语言目标，例如“把冷却过的苹果放进微波炉”，agent 需要浏览房间、识别对象，并通过文本 action 操作对象；性能由 goal-completion rate 衡量。该 benchmark 强调 tight step budget 下的多步规划和 grounded search。我们使用 valid-unseen set 中 134 个任务，覆盖六种任务类型：pick-and-place、pick-two-and-place、look-at-in-light，以及三种 transform-then-place 变体：heat、cool、clean。

WebShop。WebShop 是一个 Web 交互 benchmark，其中 agent 在模拟在线商店中扮演顾客。给定文本产品描述，agent 必须搜索、浏览产品页、选择最匹配的商品并购买；评分反映所选商品满足请求的程度。我们用固定 seed 抽样 100 个实例，每个实例作为独立 shopping session 运行。

τ^3 -Bench。 τ^3 -Bench 是多轮对话 benchmark，agent 扮演客服助手，必须在遵守显式 domain policy 的同时满足用户请求。性能通过整段对话的 rule compliance 衡量。该 benchmark 强调 dialogue-policy adherence：harness 必须防止 agent 在多轮对话中同意不允许的 action。评估时，我们选择三个 domain：Retail、Airline 和 Telecom。

SWE-bench Verified。SWE-bench Verified 是 SWE-bench 的人工验证子集，每个任务都要求 agent 通过编辑相应 repository 解决真实 GitHub issue，使项目隐藏测试套件通过。该 benchmark 强调 repository-level code editing：浏览大代码库、定位相关 fault、实现 patch、避免已有测试退化。评估时，我们从 SWE-bench Verified 抽样 55-task subset，用 patch resolution 衡量性能。

A.2 Evaluation-Set Design

表 3 中的 sampled-task count 表示每轮演化都评分的固定 evaluation set。GAIA 使用跨三个难度层级抽取的固定 103-task set，39/52/12。ALFWorld 使用 valid-unseen split 的全部 134 个任务。WebShop 用固定 seed 随机抽样 100 个任务，每个任务作为独立 shopping session。 τ^3 -Bench 选择三个 domain，Retail、Airline、Telecom，并对每个 selected domain 内完整 task list 评分。软件工程使用从 SWE-bench Verified 抽样的 55-task subset。每个 benchmark 的同一 evaluation set 在每轮重新评分，因此附录中的曲线衡量固定任务集上的逐轮变化，而不是移动样本。

A.3 Metric Definitions

Pass@ k 。对于在 task set D 上评估的配置，每个任务有 n 个 rollout。令 $r_{i,j} \in \{0,1\}$ 表示任务 i 上 rollout j 的二值 outcome。令 $c_i = \sum_{j=1}^n r_{i,j}$ 为任务 i 成功 rollout 数。我们用标准无偏估计报告 pass@ k ，即抽样 k 个 rollout 至少有一个解决任务的概率：

$$\mathrm{Pass}@k = \frac{1}{|D|} \sum_{i=1}^{|D|} \left(1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}} \right)$$

所有演化曲线都用 pass@2 作为 primary metric：每个任务有两次独立 rollout，任一成功就算 solved。这降低了对 single-rollout stochasticity 的敏感性，同时保留严格 task-level criterion。因基础设施失败而终止的 rollout，例如 sandbox crash、API timeout，计为失败，而不是剔除，以保持和官方 leaderboard protocol 可比。

A.4 演化协议和超参数

表 8 详列演化算法使用的超参数。Round 0 中，baseline 是一个 competent composed harness，并且增强了 benchmark-specific tool registry，而不是最小默认配置。因此图中展示的是相对于 competent initial harness 的增益。所有实验中 meta-agent 都是 Opus 4.6；task agent 变化为 Sonnet 4.6、GPT-5.4、Qwen3.5-9B。Per-task step limit 由对应 benchmark 决定，因为不同任务的交互长度差异很大。

表 8：演化协议超参数

Symbol	Meaning	Value
K_t	每轮 proposed candidates	4
seeds	每个 cell 的随机 seed	3
noise threshold	忽略的单轮 pass-count delta	$\pm 5\%$
$\mathcal{H}_0$	round-0 harness	Handcrafted base harness
meta-agent	驱动 Digester / Planner / Evolver / Critic	Opus 4.6
task agent	执行 benchmark task 的模型	Sonnet 4.6; GPT-5.4; Qwen3.5-9B
concurrency	parallel task rollouts	10
max-steps / GAIA		20
max-steps / WebShop		20
max-steps / ALFWorld		15
max-steps / τ^3 -Bench		200
max-steps / SWE-bench Verified		200

A.5 运行时基础设施

每个 rollout 都在 fresh environment instance 中运行，并按任务重新附加，从而防止 side effects 泄漏到任务之间，例如 WebShop cart、ALFWorld world state、shell working directory。Runtime 把每个 rollout 的完整 trajectory 记录到 observability layer：每个模型调用、工具调用和 environment observation；随后 Digester 压缩这些记录；cross-round ledgers 由该日志聚合。Task rollout 并发为 10。Meta-agent 以并发 4 运行，每个 role 的 step limit 为 200。协同演化 model training 使用 $8 \times H100$ GPUs，batch size 256，learning rate 1×10^{-6} 。

B. Prompts 和 Harness Defaults

本附录复现驱动 AEGIS outer loop 的 prompts 和 Round-0 task-agent defaults。下面 block 是生成实验结果的 commit 中对应 repository 文件的 literal contents。以下中文译文保留文件路径、模板变量和字段名。

B.1 Meta-Agent Prompts

planner/system_prompt.md

```
# Planner -- Round {{ round }}

##### `landscape.md`##### Evolver ##### cross-trace synthesis
layer##### Digesters ##### per-task overview#####

## landscape #####

- ##### digests ##### grouping##### exact-string matching #####
- ##### journal.md#####data/ship_outcomes.json#####data/rejected_candidates.jsonl#####archive/#####
outcome #####
- ##### data/task_history.jsonl#####
- ##### ship ##### regression##### R{{ round }}/regressions.md#####k-aware ##### pass-
state ##### R{{ round_minus_1 }} ##### joint-suspect ships#####ship_outcomes.json ##### hit-rate #####
predicted-task improvements##### collateral damage #####regressions.md #####
regressions##### landscape ##### "## Regressions to address" section##### responsible ship
bucket##### regression ##### unattempted_directions##### Evolver ##### first-class targets#####
- reputation signal ##### mutation layer ##### proposed -> shipped, window#####{{
reputation_summary }}
- scoreboard.json ##### ship_outcomes.json ##### bucket ##### hit rates##### bucket ##### shipped 3+ ##### hit
rate ##### bucket ##### digests ##### failure##### bucket ##### cluster#####
mutation##### Evolver #####

{% if round >= 2 %}- ##### Prior Critic ##### strategy_concern##### R{{ round_minus_1 }}/decision.md#####
frontmatter ##### strategy_concern ##### landscape #####Critic ##### strategy_concern
##### Evolver##### Evolver ##### landscape.md#####
{% endif %}

#####“budget exhaustion keeps hitting X tasks”##### digests ##### trajectory anchors#####Evolver
#####

##### coherent directions ##### overwhelming signal ##### candidates#####

## evidence #####

Run root ##### INDEX.md#####overview.md#####digests/<task_id>.md#####journal.md#####data/*.jsonl#####data/
ship_outcomes.json#####archive/##### reading list##### synthesis #####

## Output

##### `write_tool` ##### Markdown##### YAML frontmatter##### Evolver #####

---
round: {{ round }}
top_themes:
  - <theme-1>
persistent_failures:
  - <task_id>
unattempted_directions:
  - <short description>
---

## Landscape
<#####>
```

evolver/system_prompt.md

```
# Evolver -- Round {{ round }}
```

```

##### shipping ##### benchmark pass rate ##### K >= 1##### speculative
##### evidence-driven##### raw traces ■ digests■

## #####

##### research##### maintenance##### pipeline ■■ ship ■■■ bucket#####
harness ##### structural lever#####runloop ##### processor-hook ##### bucket ■
reputation ##### bucket history■gate-rejection fear ■ implementation discomfort #####

[... strategy-concern relay and revert/improve-prior-ship rules truncated ...]

## Action space #####

Mutation space ■ runtime■reachable web ■■■ harness capability set #####package■API
endpoint##### tool##### hallucination##### `bash`■`web_search`■`web_fetch` ■■■
capability ##### `capability_evidence`■

## Build -> verify -> iterate#####

##### executable code ##### manifest ##### session ##### loop■

1. ##### scratch dir■
2. #####
   - Level 1■unit call works##### processor/tool##### async hook■assert ■■ state mutation ■■■
   - Level 2■round-trip reaches the model■unit call ##### agent ##### assert
   #####tool ■ provider serializer■processor ■ next pipeline stage■
3. ##### bug##### pivot##### try/except ■■
4. ##### `capability_evidence`■"#####"#####

##### candidate##### ship slot##### flips■ prompt-bucket candidates ■■■counterfactual
gate ■■■■ smoke check■

[... reading list and write locations truncated ...]

## Manifest shape

■■ candidate ■ `{ candidates_dir }}/C-R{{ round }}-<NN>.md` ##### manifest##### applied `config.
yaml` ■ scratch dir■

---
candidate_id: C-R{{ round }}-<NN>
bucket: <prompt|tools|config|processor>
iterates_from: <prior_ship_id>
capability_evidence:
  - type: <python_package|http_endpoint|builtin_tool|filesystem|other>
    claim: "<this candidate depends on>"
    evidence: "<■ session ##### + #####>"
file_changes:
  - {path: <under scratch dir>, action: <create|modify|delete>, diff_summary: "<one line>"}
predicted_impact:
  tasks_will_unlock:    [<ALL_FAIL -> expect >=1 rollout to pass>]
  tasks_will_stabilize: [<PARTIAL_PASS -> expect all rollouts to pass>]
  tasks_at_risk:        [<currently >=1 pass -> might regress>]
attribution_signature:
  type: <tool_call|processor_invocation>
  tool_name: <PascalCase name as registered>
  expected_min_calls: 1
---

## Failure Evidence
■■ candidate ##### trajectory ■ digest anchor■

## Root Cause
## Targeted Fix
##### mutation ##### hooks / event fields / state slots / config entries■■■ Critic #####

## Why this won't break tasks_at_risk

[... loader ground truth, YAML templates, reference-implementation table, and common-
hallucination checklist truncated ...]

#### critic/system_prompt.md

# Critic -- Round {{ round }}

```

```

#####

## Part 1 -- Per-candidate verdict

##### pass rate ##### candidate##### bucket-disjoint candidates#####no-op#ship #
candidate #####

## verdict ##### candidate-vs-config interaction### candidate # `## Targeted Fix`##### hooks /
event fields / state slots##### HarnessConfig#Verdict ##### candidate # mutation surface ###
parent config ### processor#tool#prompt clause # config kwarg ##### (a)
#####candidate applied YAML ##### (b) ##### verdict ##### ask-more#

[... round-trip (Level-2) evidence check for tool/processor candidates truncated ...]

## Part 2 -- Portfolio audit

##### individual candidate ##### scoreboard.json#data/ship_outcomes.json#

- ##### lever item ### 3 ## >=2 # shipped##### hit_rate < 0.4##### ship ### lever # candidate#####
strategy_concern### miss ### k-sampling noise#####
- ##### bucket # cluster ### Evolver ##### digests/ # failure pattern ##### lever#####
- ### regressions.md #####Evolver ## ship ##### regression # candidate### "## Why this
regression is acceptable"#####no-op##### missed task IDs#

##### decision.md frontmatter ### strategy_concern### bucket#round range#hit rate#failing
tasks##### Planner ##### Evolver##### Evolver strategy #####

[... independence rule, available-to-read guide, ask_evolver, and loader ground truth truncated
...]

## Output

##### candidate### `verdicts/V-<candidate_id>.md`#

---
candidate_id: <C-R{{ round }}-NN>
verdict: <accept|reject|ask-more>
evidence_anchors:
  - trajectories/<file>#step_N
---

## Reasoning
<##### verdict### anchors#2-4 #####>

##### `decision.md`#

---
round: {{ round }}
decision_type: <ship|no_op>
ship_ranking:
  - candidate_id: <C-R{{ round }}-NN>
strategy_concern: |
  <##### ship_outcomes / task_history anchors>
---

## Reasoning
<3-6 # bullet### verdict file ##### strategy_concern#>

Multi-ship#Stage 4 ### ship ### candidate##### bucket ##### ship ### candidate##### failure
mode # bucket-disjoint candidates ##### ship### decision.md ##### ship#

```

B.2 Round-0 Task-Agent Prompts

Composition-layer default $\mathcal{H}_0$ 每个 benchmark 加载一个 system prompt。下面复现 ALFWorld default 作为代表，其余 benchmark defaults 结构相同并列在 repository 中。

```
#### alfworld_evolver/systemprompt.md
```

```
# System
```

```
### ALFRED Embodied Environment ##### expert agent##### `act` tool #####
```



```

capability_evidence:
  - {type: python_package | filesystem | other, claim: "...", evidence: "..."}
file_changes:
  - {path: "...", action: create | modify | delete, diff_summary: "..."}
predicted_impact:
  tasks_will_unlock: [task_id, ...]
  tasks_will_stabilize: [task_id, ...]
  tasks_at_risk: [task_id, ...]
attribution_signature:
  type: processor_invocation | tool_call | prompt_feature
  tool_name: <name>
  expected_min_calls: <int>

```

C. 一个演化步骤的解剖

为具体说明 AEGIS loop，本文走查一个完整 cycle：从 Digester compression，到 Planner synthesis、Evolver editing、Critic judgment，再到产生的 trace delta。我们选择 GAIA / Sonnet 4.6 run 的第 10 轮：一次覆盖新工具、prompt addition 和 configuration change 的 composite edit。这次 multi-component intervention 产生了该 run 中最大的单轮增益，比 single-lever edit 更能说明问题。

C.1 Worked Example : GAIA / Sonnet 4.6 , Round 10

Failure evidence。R10 前，success rate 为 74.8%，此前由于 R9 regression 从 77.7% 峰值下跌。Digester 的 trace analysis 揭示了一个系统性 failure pattern：R10 traces 中每次 Wikipedia fetch 都返回 0 chars。WebFetch 在网站需要 JavaScript support 时使用 browser，但 Wikipedia 新 frontend 无法正确加载，要么 timeout，要么返回空 body。Traces 很清楚：任务 db4fd70a 中，铁路线路 station 数量，db4fd70a_r0.jsonl#step_0 和 #step_1 报告 Wikipedia WebFetch 返回 0 chars；任务 f0f46385 中，ASEAN members membership status，连续三个 WebFetch calls 返回 0 chars；本轮十次独立尝试返回空响应。

Planner synthesis。Digester 按 failure mode 分组 23 个 failed tasks，浮现一个关键 tool-level issue，且 R9 Critic 已经标记过：尽管 source-access failures 自 R1 就出现，tools component 已连续九轮没有 ship 修复。Planner 收到两个目标：第一，解决持久的 tool-level source-access failures；第二，revert 导致 R9 regression 的 prompt 和 budget-processor changes。

Evolver edit。Evolver 提出 C-R10-02，覆盖三个 bucket。第一，tools：新的 WikiTextFetch tool，绕开 browser，直接使用 MediaWiki API endpoint；返回 article 全文 plain text，铁路线路 case 为 10,529 chars，ASEAN 为 80,028 chars。第二，prompt：在 tool usage section 加一句，指示 agent 查 Wikipedia article 前先用 WikiTextFetch。第三，config：恢复 round-8 baseline configuration，注册 WikiTextFetch，移除有问题的 budget processor。Manifest 的 capability evidence 包含 Level-2 round-trip check，provider 将内容序列化为 10,529-char string；attribution signature 要求至少一次 WikiTextFetch call。

R10/candidates/C-R10-02.md

```

candidate_id: C-R10-02
bucket: [tools, prompt, config]
capability_evidence:
  - type: http_endpoint
    claim: "MediaWiki API returns full plain-text extract where WebFetch returns 0 chars"
    evidence: "GET .../w/api.php?...&plaintext=true -> 10,529 chars for Franklin/Foxboro_Line"
  - type: other

```

```

    claim: "tool return survives provider serialization to the model (Level 2)"
    evidence: "_prepare_messages([tool_msg]) keeps content as 10,529-char string"
file_changes:
  - {path: R10/applied/C-R10-02/wiki_text_fetch.py, action: create, diff_summary:
"WikiTextFetch via MediaWiki API"}
  - {path: R10/applied/C-R10-02/gaia_agent.md, action: create, diff_summary: "R8 prompt + one
WikiTextFetch line"}
  - {path: R10/applied/C-R10-02/config.yaml, action: create, diff_summary: "register tool;
restore R8; drop budget processor"}
predicted_impact:
  tasks_will_unlock: [db4fd70a, f0f46385, 983bba7c, 08f3a05f, 5e2a91b0]
  tasks_will_stabilize: [4b6bb5f7, 42d4198c]
  tasks_at_risk: []
attribution_signature:
  type: tool_call
  tool_name: WikiTextFetch
  expected_min_calls: 1

```

Critic verdict。Critic 只批准 C-R10-02，拒绝竞争的 revert-only candidate C-R10-01。理由覆盖三点：第一，interaction：C-R10-02 是 C-R10-01 的严格超集；两者都恢复 round-8 baseline，因此 budget-processor removal 是有意的，不是意外重叠。第二，round-trip evidence：Critic 在接受任何 tools-bucket candidate 前验证 Level-2 evidence，即 tool output 以完整 string 到达，而不是 truncation marker。第三，portfolio：这是十轮中的第一次 tools-bucket ship，持久 zero-char returns 的 trace evidence 证明了 R9 已标记 intervention 的必要性。

Delta realized。Shipping 后，GAIA pass rate 从 R9 的 74.8% 提到 R10 的 79.6%，+4.9pp，5 个任务变为 pass，这是整个 run 中最大提升；该工具预测会影响的 7 个任务中有 5 个翻转为 pass，hit rate 0.71，是全部 19 个 run 中任何 ship 的最高 hit rate。改进主要发生在 Level 2，+4 tasks，和 Level 3，+2。由于工具在目标任务上触发，attribution condition 被满足。

图 7 用 manifest card 总结同一个 edit：上方 raw YAML 是 loop 记录的内容，card 是其面向人的解读。

图 7：C-R10-02 manifest card

区块	中文翻译
Failure evidence	本轮 trajectories 中每次 Wikipedia WebFetch 都返回 0 chars，跨 db4fd70a、f0f46385、42d4198c 超过 10 次调用；headless browser fallback 在 Wikipedia frontend 上 timeout。
Edit	新 WikiTextFetch tool 调用 MediaWiki API 获取 article plain-text content；一条 prompt line 将 Wikipedia lookup 优先路由到它；恢复 round-8 baseline 并移除造成 regression 的 budget processor。
Predicted fixes	Unlock db4fd70a、f0f46385、983bba7c、08f3a05f、5e2a91b0；stabilize 4b6bb5f7、42d4198c。
Attribution signature	WikiTextFetch 上的 tool_call，至少 1 次；用下一轮 trace 验证。

图注翻译：C-R10-02 的 change manifest 被渲染为 manifest card，即上方 logged YAML 的人类可读对应物。

D. 附加结果

附录剩余部分按 benchmark 组织。每个 subsection 围绕一个三 panel figure 展开：a 是 adaptation loop 需要处理的 failure cluster breakdown，b 是每个模型 shipped harness lever 的分布，c 是 model-by-lever heatmap，显示每个 lever 的 effectiveness，即 tasks flipped to pass / tasks

predicted。阅读顺序是：什么失败以及为什么；每个模型如何演化；演化是否关闭了失败。

D.1 GAIA

GAIA 强调 tool use 下的一般推理，是本文 benchmark 中 lever 最多样的。图 8 给出本附录贯穿使用的三种视图。

Failure clusters and causes。Panel (a) 总结 GAIA run 中累计的 failure clusters。主导 cluster 是 blocked-source，39%，即 agent 无法检索所需证据，因为页面返回空内容、需要会 timeout 的 JavaScript rendering，或信息不完整。Reasoning failures 其次，占 33%，包括需要 multi-hop inference、相似实体 disambiguation、数值计算，或精确解释 underspecified queries 的任务。Figure/visual failures 占 11%，出现于答案依赖图片、地图或图表中信息，而 text extraction 无法捕获。Document/table parsing failures 占 11%，出现于证据锁在 PDF、结构化表格或 semi-structured format 中，agent 错解 layout 或漏掉相关 cells。Scope ambiguity 占 6%，覆盖存在多个 valid interpretation 的 query，agent 回答了相关但错误的解读。整体上，GAIA 失败集中在 evidence retrieval、multi-step reasoning、visual grounding、structured-document extraction 和 query disambiguation。

Per-model evolution logic。Panel (b) 显示 GAIA 是唯一四种 lever 都大量使用的 benchmark；仅 Sonnet run 就 ship 了 11 个 prompt、7 个 processor、7 个 config、6 个 tools edit，因为其 failure set 同时跨 tool、prompt、config 问题。三个模型仍按起始能力分化：Sonnet 轮次最多，扫过所有 lever；GPT-5.4 最依赖 prompt，45% ships，几乎不碰 config，因为其 reasoning 已足够强，剩余增益多是 instruction-following；Qwen3.5 短 run 中 few ships 很集中，尤其单个 tools ship 产生所有 cell 中最高 yield。共享逻辑是 behavioral cluster 用 prompt-first，而把稀缺 tools lever 留给 prose 无法解决的 mechanical cluster。

演化是否关闭 cluster。Panel (c) 显示演化减少了哪些 failure cluster。最大改进来自 blocked-source cluster：引入 WikiTextFetch 的 tool edit 用 MediaWiki API call 替代不可靠的 browser-based Wikipedia fetching，减少空页面或不完整页面检索造成的失败。Prompt edits 主要针对 reasoning cluster，鼓励更显式 verification，并带来稳定提升。相比之下，figure/visual 和 document/table parsing failures 更难减少，因为它们需要从 images、figures、PDFs 或 structured tables 中抽取信息。总体上，GAIA 通过 tool edits 修复 retrieval failures、通过 prompt edits 减少 reasoning errors；残余错误集中在 visual 和 document-heavy tasks。

图 8 : GAIA evolution analysis

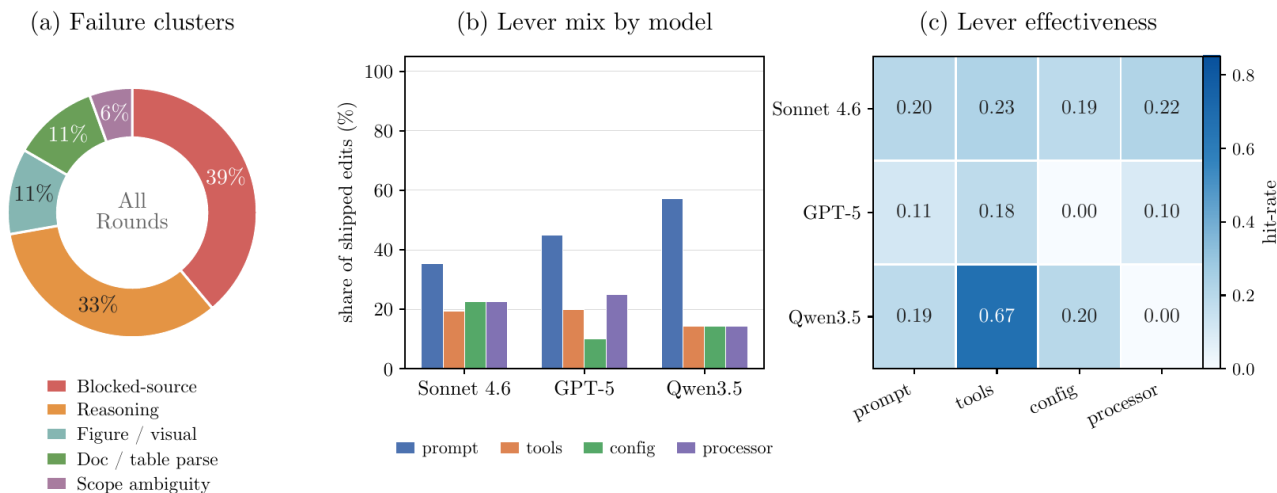


图 8 原图；图内英文已在下方表格翻译为中文。

图内文字翻译：

原文	中文
Failure clusters	失败簇
Lever mix by model	按模型划分的 lever mix
Lever effectiveness	Lever 有效性
Blocked-source	来源受阻
Reasoning	推理
Figure / visual	图像 / 视觉
Doc / table parse	文档 / 表格解析
Scope ambiguity	范围歧义
share of shipped edits (%)	shipped edits 占比
hit-rate	命中率
prompt / tools / config / processor	提示词 / 工具 / 配置 / processor

D.2 ALFWorld

ALFWorld 是 embodied planning benchmark，也是本文最 prompt-dominated 的 benchmark。图 9 展示其 clusters、per-model logic 和 effectiveness。

Failure clusters and causes。Panel (a) 总结 ALFWorld 的主要 failure clusters。主导 cluster 是 search / step-ceiling，89%，包括 agent 搜索房间或 receptacle 顺序低效，或者在完成 deep search 或 transform-then-place 等长交互链前达到 step limit。Prompt-rule side-effect failures 占 7%，出现于新增 heuristic 改进一些任务，却无意限制另一些行为，导致 agent 跳过 valid action 或过早停止搜索。Object-type confusion failures 占 4%，指 agent 混淆语义相似对象。整体上，ALFWorld 失败主要来自 search efficiency、over-constrained prompting 和 object-specific grounding errors。

Per-model evolution logic。Panel (b) 显示 prompt dominance 与 base-model strength 反向 scaling：prompt rule 只对能可靠遵循它的模型产生收益。Sonnet 是最强 base，几乎所有改进都来自

prompt edits ; system prompt 中的 search-order heuristics 足够，因为它能一致遵守。GPT-5.4 用 processor 补充 prompts，该 processor 在 R3 引入，用于管理 transformation tasks 的 step budget。Qwen3.5 需要最多样的 mix，prompt、config、processor，包括一个 processor，拦截其 reasoning text，并在需要时重新 emit tool calls；这是 prompt-level steering 无法解决的 mechanical fix。共享模式是 prompt-first；只有 prompt 不足时才招募 structural levers。base model 越弱，演化越早从 prompt-based steering 回退到 config 或 processor enforcement。

演化是否关闭 cluster。Panel (c) 显示演化减少了 ALFWorld 的主要 failure clusters，但不同 task agent 依赖不同 lever。对 Qwen3.5，processor 和 config edits 效果最强，hit-rates 分别为 0.84 和 0.71。这些 edit 直接处理 mechanical failures，例如重新 emit missed tool calls、调整 execution budgets，使 Qwen3.5 提升 +44.0pp 并接近 closed-model runs。对 Sonnet，剩余失败较少结构性，因此 prompt edits 足够带来多数收益，hit-rate 0.49，而 processor edit 效果边际，0.14。两个 cluster 只被部分减少：prompt-rule side effects，它们由某些 evolved heuristics 引入并在后续轮修补；以及 long-path failures，一些 episode 仍超过可用 interaction budget。总体上，ALFWorld 展示清晰的 model-dependent pattern：强模型主要受益于 prompt-level steering，弱模型需要更多 processor 和 configuration edits 提供结构支持。

图 9 : ALFWorld evolution analysis

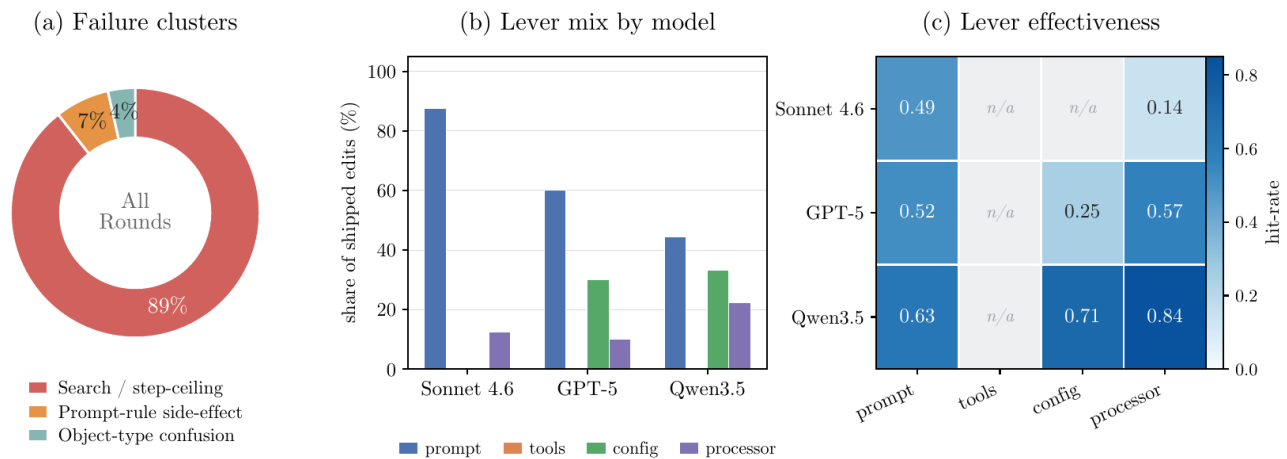


图 9 原图；图内英文已在下方表格翻译为中文。

图内文字翻译：

原文	中文
Search / step-ceiling	搜索 / 步数上限
Prompt-rule side-effect	提示词规则副作用
Object-type confusion	对象类型混淆
Lever mix by model	按模型划分的 lever mix
Lever effectiveness	Lever 有效性

图注翻译：ALFWorld 演化分析，134 个任务，goal-completion。a 为跨所有轮次累计的 failure clusters；搜索低效和硬 step-ceiling 主导，另有两个小 cluster，分别是演化自身引入的 prompt-rule side-effect 和短暂出现的 object-type confusion。b 为按模型划分的 lever mix：强 base Sonnet 几乎只靠 prompt 攀升，较弱 base 使用更多样 lever。c 为 lever effectiveness：structural levers，processor 和 config，在弱模型上使用更多且更有效。

D.3 WebShop

WebShop 是 Web 交互 benchmark，也是本文噪声最大的 run。图 10 展示其 clusters、per-model logic 和 effectiveness。

Failure clusters and causes。Panel (a) 总结 WebShop run 中累计的 failure clusters。早期失败由 search and pagination loops 主导：agent 反复重写 query 或在结果页中循环，而不 commit 购买。随着演化减少这些 control-flow errors，剩余失败转向 product-selection judgment。最大 cluster 是 wrong product，46%，发生于 agent 选择错误品类的商品，或在比较替代项前接受 weak match。Pagination loop failures 占 21%，捕捉剩余的反复 next/previous navigation 且无进展。Colour matching failures 占 17%，出现于 agent 错处理 shade equivalence 或站点特定 color labels，例如把 “wine” 和 “red” 当作不兼容。Attribute check failures 占 17%，发生于所选 item 接近请求但某个必需细节不满足，例如 size、sleeve length 或其他未验证属性。整体 cluster shift 表明，演化先减少 navigation loops，随后主要错误集中在 product matching 和 attribute verification。

Per-model evolution logic。Panel (b) 显示 prompt edits 驱动三个模型的大多数改进，processor edits 作为稳定的 secondary lever。这一模式匹配 WebShop 的主要 control-flow failures。Prompt rules 帮助 agent 更高效搜索并更早 commit；advisory processors 在执行中强化这些规则，当 agent 开始重复搜索或在 pagination 中循环时添加 warning。对 product-selection failures，演化引入更 targeted support：colour-matching tool 帮助处理 shade-equivalence cases，config edits 帮助弱模型在更长 shopping session 中维持 context。总体上，WebShop 需要混合响应：prompts 改进高层 shopping strategy，processors 减少 navigation loops，tools 支持 attribute matching，config edits 稳定 long-session behavior。

演化是否关闭 cluster。Panel (c) 显示演化部分减少了 WebShop failure clusters。Prompt edits 是跨模型最稳定有效 lever，hit-rates 0.37-0.50；config edits 帮助两个弱模型在更长 session 中维持 context。这些变化减少了早期 search 和 pagination loops，使性能从 60% 提到 76% 峰值。剩余 clusters 更难关闭。Advisory processors 只带来 modest gains，0.20-0.25，因此部分 pagination failures 持续存在。Colour-matching tool 在本 run 中没有提升表现，hit-rate 0.0，因此 colour-matching cluster 基本未变。总体上，WebShop 最受益于 prompt 和 config edits，残余 navigation loops 与 product-judgment errors 仍是主要不稳定来源。

图 10 : WebShop evolution analysis

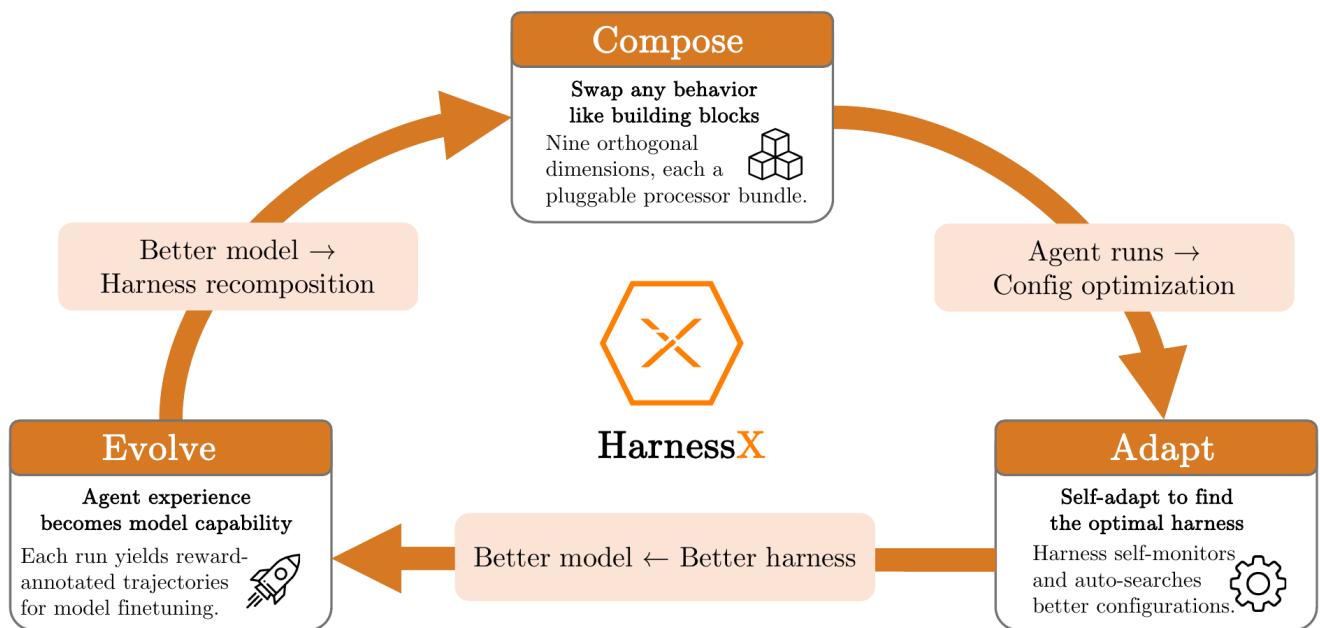


图 1 原图；图内英文已在下方表格翻译为中文。

原文	中文
Wrong product	商品错误
Pagination loop	翻页循环
Colour matching	颜色匹配
Attribute check	属性检查
prompt / tools / config / processor	提示词 / 工具 / 配置 / processor

D.4 τ^3 -Bench

τ^3 -Bench 强调显式 domain policy 下的多轮对话。图 11 汇总 airline、retail 和 telecom domain 上的 AEGIS runs。

Failure clusters and causes。排除 harness-interrupt traces 后，失败以 judgment-heavy 为主：前两大 clusters，premature / unverified action，28%，即在 precondition 成立前提交 booking、refund 或 device fix；wrong selection / count，24%，二者合计超过一半，关心何时 commit 和选什么，而不是 mechanical execution。剩余是 procedural，incomplete multi-step fix 16%，missed step / sub-task 14%，或 policy-related，misinterpretation 13%。最小 cluster 是 capability-boundary confusion，5%，这是 τ^3 特有的：某些 telecom fault 发生在用户 handset 上，agent 没有 device-side tool，失败是把边界误当成缺失 capability。

Per-model evolution logic。每个模型的演化都由 prompt 和 processor 驱动，zero tools edits：tool set 固定，没有哪个 cluster 能由新工具关闭。Sonnet 4.6 在 prompt/processor 上分配 23/18；GPT-5.4 mix 最平衡，19/20；Qwen3.5-9B ship 总量更少，14/9。由于 τ^3 failures 是 control-flow 和 judgment errors，productive levers 是编码 policy ordering constraints 的 prompt rules，以及在对话中强制执行它们的 processors。

演化是否关闭 cluster。Config 在使用时最锋利，Qwen3.5 hit-rate 0.67，GPT-5.4 0.33，但 ship 很少；高量 prompt 和 processor levers 中等有效，0.27-0.35，与 dominant clusters 的 control-flow 本质匹配。Gains 跟 base-model headroom 一致：GPT-5.4 起点最低，76.2%，提升最大 +14.5pp；Sonnet 4.6 提升 +5.4pp；接近天花板的 Qwen3.5-9B 只提升 +1.1pp。Loop 不单调：Sonnet telecom run 在 R4 达到 100%，第六个连续 same-bucket edit 后 R7 退化到 80.7%，随后 R9 恢复到 99.1%。总体上，ordering-enforcing levers 最可靠地关闭 premature-action 和 missed-step clusters，而 wrong-selection 和 policy-judgment errors 是更难的残余。

图 11 : τ^3 -Bench evolution analysis

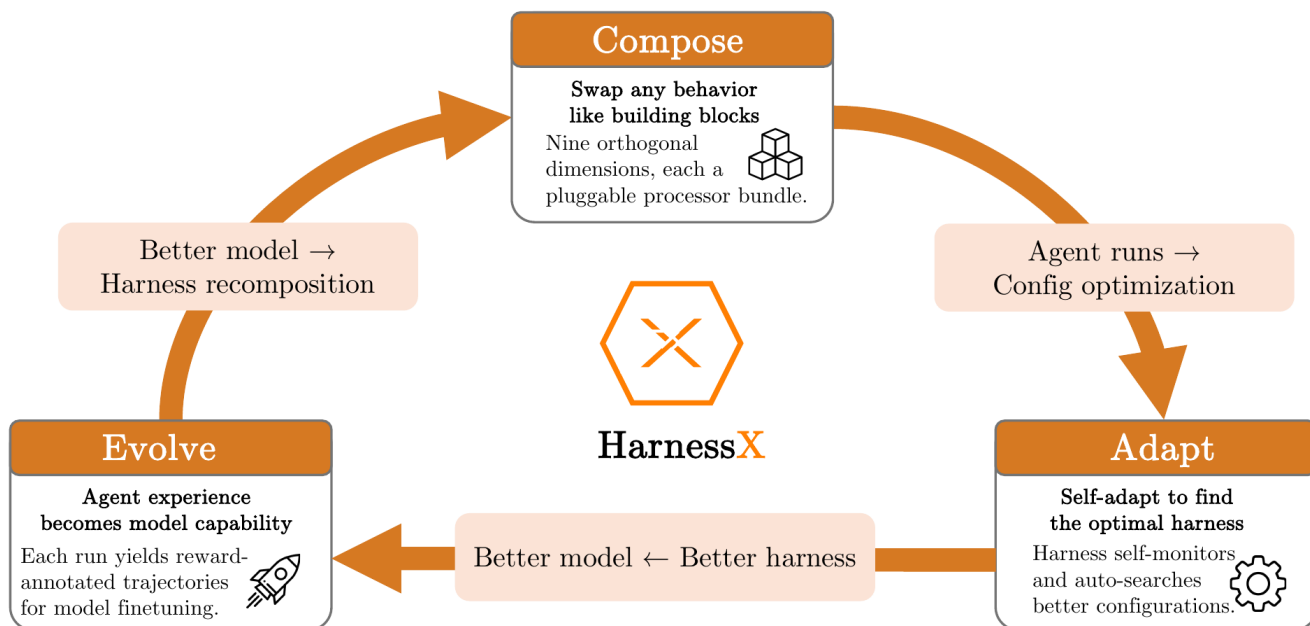


图 1 原图；图内英文已在下方表格翻译为中文。

原文	中文
Premature / unverified action	过早 / 未验证 action
Wrong selection / count	错误选择 / 数量
Incomplete multi-step fix	不完整多步修复
Missed step / sub-task	漏步骤 / 子任务
Policy misinterpretation	策略误读
Capability-boundary confusion	能力边界混淆

D.5 SWE-bench Verified

SWE-bench Verified 强调 repository-level code editing。图 12 展示其 clusters、per-model logic 和 effectiveness。

Failure clusters and causes。Panel (a) 汇总跨所有轮次和三个模型的 failure clusters。主导 cluster 是 incomplete fix，62%，agent 找到了正确区域并产生 valid patch，但只覆盖一个 branch 或 call site，而 gold patch 需要多个。Wrong diagnosis 占 19%，包括误读 root cause 后编辑错误文件或抽象层。剩余尾部更多是 mechanical 而非 cognitive：no edit attempted 6%、Edit anchor mismatch 5%、budget exhausted 4%。值得注意的是，该构成和 reward hacking 相反：失败是 under-fixes，不是 gamed evaluations，因为 harness 会先应用 gold test patch 再应用 model patch，并阻止写 test files。

Per-model evolution logic。Panel (b) 显示 SWE-bench 对每个模型都是 prompt-first，secondary lever 跟 base-model strength 相关。三个 run 都 ship zero tools edits，因为不同于 GAIA，failure set 没有能由工具关闭的 mechanical-retrieval cluster。Sonnet 把 prompt 和 processor edits 平分，7 个各自，用 workflow nudges 塑造已有能力的 coder；GPT-5.4 最依赖 prompt，8 ships，并用 config，4 ships，revert harmful nudge 并重构 strategy phases；Qwen3.5 的 few ships 分散在 prompt、processor、config，6/3/3。共享逻辑是 prompt-first，随着 base model 变弱才招募

structural levers。

演化是否关闭 cluster。Panel (c) 展示出清晰 capability floor。对强模型，productive levers 确实关闭失败：GPT-5.4 的 config edits 达到 0.48，prompt 0.39；Sonnet 的 prompt 和 processor 都为 0.40。对 Qwen3.5-9B，每个 lever 都接近零，prompt 0.05、config 0.05、processor 0.06，低一个数量级，因为 9B base 无法执行 predicted fixes。同一个 loop 可以把 GPT-5.4 从 45% 拉到 64% peak，并把 Sonnet 稳定在约 87%，但对 Qwen3.5 只有噪声，peak 42%，zero durable gains。总体上，SWE-bench 通过 broaden fix scope 的 prompt edits 和 restore workflow pacing 的 config edits 改进，但只对强到足以执行它们的模型有效。

图 12 : SWE-bench Verified evolution analysis

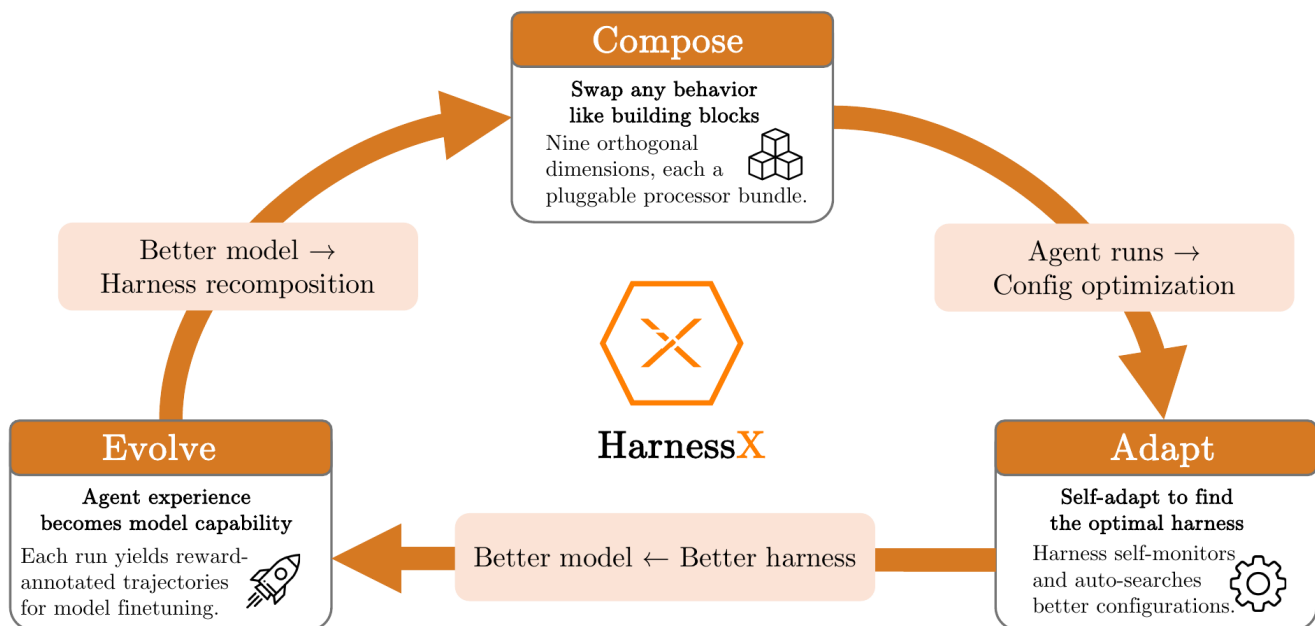


图 1 原图；图内英文已在下方表格翻译为中文。

原文	中文
Incomplete fix	修复不完整
Wrong diagnosis	诊断错误
No edit attempted	未尝试编辑
Old string mismatch	旧字符串不匹配
Budget exhausted	预算耗尽
Wrong file	文件错误
Test-verify loop	测试-验证循环
Other	其他
hit-rate (flips / predicted)	命中率（翻转 / 预测）

E. 可复现性和 Artifact

E.1 Per-Run Directory Layout

每个 evolution run 写出一个自描述目录。下面 layout 使读者能从 logged artifacts 重构本文中的任何 decision。

```
runs/<run_name>/
|-- INDEX.md           # ■■■■■ run index
|-- journal.md         # ■■■■ memo■■■■■ entry
|-- curves.json        # per-round pass-rate trajectory
|-- scoreboard.json    # ships + per-bucket reputation
|-- audit.jsonl        # ■■■■ event log (stage / gate / commit)
|-- data/
|   |-- task_history.jsonl # ■■ (round, task) ■■
|   |-- ship_outcomes.json # ■■ historical ship ■■ entry
|   |-- rejected_candidates.jsonl
|-- R<n>/              # per-round artifacts
|   |-- landscape.md     # Planner cross-trace synthesis
|   |-- candidates/C-R<n>-NN.md # Evolver change manifests (■■ K ■)
|   |-- applied/C-R<n>-NN/ # applied config + asset files
```

```
|-- decision.md          # Critic ship / no_op decision
|-- verdicts/V-C-R<n>-NN.md # per-candidate verdicts
|-- regressions.md       # ■■ R<n-1> ■■■■■■
|-- digests/*.md         # per-task failure analysis
`-- trajectories/*.jsonl # raw rollouts
```

图表文字译表补充

图 6 九宫格失败案例



图 6 原图；图内英文已在下方表格翻译为中文。

Panel	原文要点	中文
a	Reward hacking (GAIA, Sonnet 4.6)	R10 WikiTextFetch ship, +4.9pp, 5/7 flips; R11 trace 显示部分任务通过格式 exploit, 而非 cross-check; R12 guard 要求第二条 retrieval path。真实检索增益和 verifier exploit 在 R10 同时 ship; trace 在 R11 抓到 exploit; R12 guard 移除 hacking path。
b	Reward hacking (WebShop, Sonnet 4.6)	R1 anti-loop ship, success rate +16pp、reward +0.16; R5 reward 提升但 success rate 持平; R7 解耦达到峰值, reward 0.87 对 success rate 0.76, gap +0.113。Near-match 拿到 partial reward, mean reward 上升但 exact-match success rate 冻结在 76%。
c	Reward hacking (SWE-bench, GPT-5.4)	R5 所有 gate 通过, 但 score 20 -> 16; R9 再次 gate 通过, 19 -> 14; R12 对比: workspace ship 带来真实 +3。Gate 检查 replay、novelty、structure, 而不是 pass rate; 合规 ship 仍会 regression。
d	Forgetting (τ^3 -Bench Telecom, Sonnet 4.6)	R2-R6 五个 same-bucket edits, R4 达 100%; R7 第六个 edit 造成规则耦合, 94.7 -> 80.7, -14pp; R8-R9 ensemble 恢复到 99.1%。Critic 标记 bucket concentration 但仍 ship; pass@2 漏掉 coupling。
e	Forgetting (SWE-bench, GPT-5.4)	R2-R3 上升到约 61.8% peak; R4-R5 post-peak drop 到约 50.9%; R9 Verification Gate ship, 破坏 2 个有 9-round P-streak 的任务; R10 追逐 phantom regression, 恢复 3 个。多数 dip 下一轮恢复; catastrophic forgetting 在这里是中等 churn, 不是永久 wipe-out。
f	Forgetting (GAIA, GPT-5.4)	R1-R4 只有 advisory, R4 peak 73.8%; R5-R6 enforcement cliff, SearchBudget + ComputationGate, 76 -> 55; R12 rollback 失败, 最终 49.5%, -24.3pp。每个 enforcement edit 单独通过 gate, 低于阈值的累积抹掉 R4 gains。
g	Under-exploration (ALFWorld, Sonnet 4.6)	R4-R5 prompt-only edits, 增益停滞, 每轮低于 1pp; R6 唯一 processor edit, 14%, 1/7, 随后 reverted; R7-R8 ship-prediction 到 0, 然后 no_op, plateau 94.8%。Prompt space 在 R4 耗尽; 结构 edit 失败并回滚; ship-prediction 80% -> 0%。
h	Under-exploration (GAIA, GPT-5.4)	R1-R9 零 tools ships; blocked-source 39%, 只动 prompt/config; R9 Critic 标记 tools neglect 但仍未 ship; R10 WikiTextFetch +4.9pp, hit rate 0.71。Trace 从 R1 就指向 tools, pipeline 直到 R10 才 ship 结构性 tool。
i	Under-exploration (WebShop, Sonnet 4.6)	R1-R7 零 tools ships, prompt-locked, hit-rate 0.31; R7 Critic 标记 tools bucket untouched 但仍未 ship; R8 第一个 tool ship ColorMatcher, -4pp, 76 -> 72, 2/5。Pipeline 连续七轮停留在 prompt-local, 迟来的 tool ship 造成 regression。

Benchmark 图通用标签

原文	中文
Failure clusters	失败簇
Lever mix by model	按模型划分的 lever mix
Lever effectiveness	Lever 有效性
All Rounds	全部轮次
share of shipped edits (%)	shipped edits 占比
hit-rate / hit-rate (flips / predicted)	命中率 / 命中率 (翻转 / 预测)
prompt	提示词
tools	工具
config	配置
processor	processor